FACULTY OF INFORMATION TECHNOLOGY AND ELECTRICAL ENGINEERING

Martti Mourujärvi

BUILDING A SELF-SERVICE LEARNING PLATFORM USING CLOUD NATIVE TOOLS AND GITOPS

Master's Thesis
Degree Programme in Computer Science and Engineering
May 2025

Mourujärvi M. (2025) Building a Self-Service Learning Platform Using Cloud Native Tools and GitOps. University of Oulu, Degree Programme in Computer Science and Engineering, 96 p.

ABSTRACT

Providing hands-on laboratory exercises for developers is an effective way to build competence with new tools and methodologies. While many commercial and open source platforms offer learning paths for users, they often come at a cost and offer limited customization. Additionally, emerging methodologies such as chaos engineering often lack guided learning tracks altogether.

To address these gaps, this study explores the potential of using cloud native tools and methodologies in building a scalable learning platform in cloud. This study highlights how self-service learning platforms can streamline ordering of laboratory environments, reduce manual intervention by leveraging automated deployments, and automatically scale under growing demand.

This thesis is grounded in a comprehensive review of existing literature on current trends in cloud development, including the rise of cloud native practices and the adoption of open source technologies in the industry. Based on this research, a scalable learning platform with self-service capabilities is developed using curated cloud native tools. A novel chaos engineering lab exercise—featuring a reference application for fault injection and chaos engineering tools absent from other learning platforms—is designed to evaluate the built platform.

The learning platform maintained stability with multiple concurrent chaos engineering laboratories running chaos experiments simultaneously. The blast radius of individual fault injections was effectively minimized through strict access controls assigned to each user, ensuring that the disruptions remained contained within their respective laboratory environments. Even under heavy load with a continuously growing number of concurrent laboratory environments, the platform auto-scaled effectively to meet the increasing demands of user activity.

In summary, this thesis provides insights into the cloud native ecosystem and demonstrates how cloud native tooling can be leveraged to build automated, self-service learning platforms. The implemented platform serves as a novel example of modern platform development using open source technologies, showcasing demand-driven auto-scaling for cost-efficient resource utilization and self-service capabilities with a high degree of automation to minimize manual intervention throughout the entire laboratory exercise.

Keywords: cloud native, containers, kubernetes, gitops, chaos engineering, backstage

Mourujärvi M. (2025) Laboratorioharjoitusten tarjoaminen itsepalveluna pilvinatiivien työkalujen ja GitOpsin avulla. Oulun yliopisto, Tietotekniikan tutkinto-ohjelma, 96 s.

TIIVISTELMÄ

Käytännön harjoitusympäristöjen tarjoaminen kehittäjille on tehokas tapa kehittää uusien työkalujen ja menetelmien osaamista. Monet kaupalliset ja avoimen lähdekoodin alustat tarjoavat jo käyttäjilleen käytännön oppimispolkuja, mutta ovat usein maksullisia tai tarjoavat rajalliset mahdollisuudet räätälöidä harjoituksia. Lisäksi orastaville menetelmille, kuten kaaostekniikalle, ei ole tarjolla lainkaan käytännön harjoitteita.

Tässä diplomityössä tutkitaan pilvinatiivien työkalujen ja menetelmien potentiaalista käyttöä skaalautuvan harjoitusalustan rakentamisessa pilveen. Tutkimuksessa korostetaan, miten itsepalveluna tarjottava ja automaatioon tukeutuva oppimisympäristö virtaviivaistaa prosesseja, vähentää manuaalista työtä ja skaalautuu automaattisesti kysynnän kasvaessa. Tämä tutkielma sisältää kirjallisuuskatsauksen modernin pilvikehityksen suuntauksista, keskittyen pilvinatiivien työkalujen kasvavaan suosioon sekä niiden hyödyntämiseen skaalautuvien alustojen rakentamisessa. Tähän tutkimukseen perustuen, toteutetaan skaalautuva harjoitusalusta itsepalveluominaisuuksilla, käyttäen valikoituja pilvinatiiveja työkaluja. Lisäksi, tässä työssä toteutetaan käytännön harjoitusympäristö kaaostekniikkaan harjoittelemiseksi, jota käytetään myös rakennetun alustan arvioimisessa.

Rakennettu harjoitusalusta säilytti vakautensa, kun kaaoskokeita suoritettiin yhtäaikaisesti useissa rinnakkaisissa harjoitusympäristöissä. Yksittäisten vikainjektioiden vaikutussäde (blast radius) minimoitiin kullekin käyttäjälle asetetuilla käyttöoikeusrajoituksilla, joilla pystyttiin varmistamaan, että ajetut häiriöt pysyivät hallitusti ja tarkoituksenmukaisesti tietyn harjoitusympäristön sisällä. Alustalle tehdyissä testeissä harjoitusalusta kykeni automaattisen skaalauksen ansiosta vastaamaan käyttäjämäärän kasvuun myös raskaan kuormituksen aikana.

Yhteenvetona tämä tutkielma tarjoaa kirjallisuuskatsauksen pilvinatiivehin työkaluihin, ja osoittaa miten näiden työkalujen avulla voidaan rakentaa itsepalvelukykyisiä alustoja harjoitusympäristöjen tarjoamiseksi.

Avainsanat: pilvinatiivi, kontitus, kubernetes, gitops, kaaostekniikka, backstage

TABLE OF CONTENTS

FOREWORD

This thesis has been the toughest endurance test of my life to date and serves as a fitting conclusion to my years of study. I want to especially thank my mom and dad for their support, my supervisors at OP and the University of Oulu and for their guidance and help, the special interest group at OP for their participation during this thesis, and everyone who offered their support or advice during the writing of this thesis.

Oulu, May 21st, 2025

Martti Mourujärvi

LIST OF ABBREVIATIONS AND SYMBOLS

RE	Reliability Engineering
SRE	Site Reliability Engineering
IaC	Infrastructure as Code
SLA	Service Level Agreement
SLI	Service Level Indicators
SIG	Special Interest Group
CD	Continuous Delivery
AI	Artificial Intelligence
MSA	Microservice Architecture
OS	Operating System
IDP	Internal Development Platform
SDK	Software Development Kit
FMEA	Failure Mode and Effects Analysis
MTTR	Mean Time to Repair
AWS	Amazon Web Services
CPU	Central Processing Unit
JVM	Java Virtual Machine
VM	Virtual Machine
K8s	Kubernetes
WSL	Windows Subsystem for Linux
IaaS	Infrastructure as a Service
PaaS	Platform as a Service
SaaS	Software as a Service
FaaS	Function as a Service
AIaaS	Artificial Intelligence as a Service
XaaS	Everything as a Service
LaaS	Labs as a Service
CSP	Cloud Service Provider
CNCF	Cloud Native Computing Foundation
ACS	Azure Chaos Studio
AKS	Azure Kubernetes Service
JSON	JavaScript Object Notation
EKS	Elastic Kubernetes Service
ECS	Elastic Container Service
ACP	Azure Container Platform
ACR	Azure Container Registry
ACI	Azure Container Instances
VCS	Version Control System
DVCS	Distributed Version Control System
CVCS	Centralized Version Control System
SCM	Source Code Management
APM	Application Performance Monitorin
RBAC	Role Based Access Control
RPN	Risk Priority Number

RCA	Root Cause Analysis
NFR	Non-functional Requirements
VPN	Virtual Private Network
DNS	Domain Name Service
CLI	Command Line Interface
CRD	Custom Resource Definition
OTel	OpenTelemetry
etcd	Distributed key-value store used by Kubernetes

1. INTRODUCTION

In this thesis, a practical, hands-on learning platform will be designed and deployed in a cloud environment, using cloud native architecture and tooling, and delivered with GitOps practices, where code in a version control system serves as the single source of truth for deploying both infrastructure and applications. The platform and first piloting laboratory exercise are designed for the use of Reliability Engineers (REs) at OP, but can be extended to the use of anyone regardless of title. During this study many cloud native tools are evaluated to be used with the platform and for the pioneering chaos engineering lab, where faults are deliberately injected into controlled environments to observe system behavior. The platform is designed as a comprehensive end-to-end environment, starting with the ordering template, and concluding with the cleanup of resources after the exercise is completed, while delivering it as a complete self-service solution for end users. These capabilities include the automated ordering and decommissioning of active laboratory environments by students. Moreover, the design of the pilot lab exercise is presented and used to evaluate the reliability, usability, and security of the built platform. In this chapter, the fundamental topics of the study are introduced to the reader, including the motivations, research questions, and constraints that will shape this study.

1.1. Background and Motivation

OP Financial Group has been an adopter and contributor to Spotify's open sourced Internal Developer Platform (IDP) project, Backstage. The platform is developed in collaboration with multiple platform teams and has become one of the key enablers of OP's large-scale cloud transformation efforts. Some of the goals of the platform are to reduce complexity in daily work, increase efficiency, and enhance the developer experience at OP.

The development teams at OP are undergoing a significant cloud transformation to support organizational strategic shift towards modern, scalable, and more resilient infrastructure. This transformation necessitates continuous learning, particularly in the domains of Reliability Engineering, to ensure the development of robust and resilient systems in the cloud.

As part of this initiative, a platform will be developed to support the transformation journey to provide a secure and reliable platform for learning new tools. More specifically, chaos engineering serves as a practical application of reliability testing applications in the cloud. This platform in the beginning be used to educate and train professionals at OP with laboratory exercise environments in the cloud.

The platform will play a crucial role in developing essential skill sets and training new and experienced professionals in various roles. The laboratories will offer hands-on experience, guided learning, and tool familiarization, ensuring that all participants gain proficiency in the necessary technologies. By facilitating laboratory environments through automation and self-service, it promotes OP's goals of scalability, reliability, and operational excellence in the cloud.

Building a learning platform with cloud native tools presents several technical challenges. One key limitation is the need to abstract away the complexity

of operating laboratory infrastructure-such as deploying environments, enforcing idempotent ordering to prevent students from launching multiple instances of the same lab, and configuring networking—so that students can focus solely on learning objectives without being overwhelmed by technical details. Additionally, ensuring robust security and isolation between student environments in a shared platform requires careful design of access controls, network policies, and resource boundaries. These considerations are critical to preventing unwanted traffic and maintaining a secure and reliable learning experience for students.

Additionally, during this thesis, a novel laboratory exercise environment will be built that enables hands-on learning with chaos engineering tools in a safe and secure way within the learning platform. The chaos engineering laboratory will also be used to evaluate the reliability and safety of the learning platform.

The following research questions were formulated to guide this thesis:

- **RQ1** What architectural patterns, tools, and best practices are necessary to design and build a scalable, reliable, and secure platform in the cloud?
- **RQ2** What kind of ordering process should be implemented to facilitate the creation, management, and decommissioning of laboratory environments, with automation and as a self-service?
- **RQ3** How costs can be effectively managed when operating ephemeral laboratories in the cloud?

1.2. Contribution

While independently progressing this study, I also presented my work and platform development at a biweekly Special Interest Group (SIG) meeting dedicated to engineers interested in improving reliability engineering practices at OP. The SIG contributed to this thesis by reviewing my work, providing feedback, and volunteering to participate in a playtest that evaluated both the platform's capabilities and the piloting lab exercise. These regular meetings offered an excellent opportunity to gather timely feedback during the thesis writing process. The SIG is open to all reliability engineers and anyone interested in advancing reliability practices at OP.

2. LITERATURE REVIEW

In this chapter, a comprehensive evaluation of the current cloud development landscape is undertaken, and relevant literature on state-of-the-art methods and tooling is identified to reflect current practices. The focus will include a critical examination of current trends with cloud computing paradigms, tools, methodologies, and key practices and metrics that enable reliability in the cloud. Through a systematic analysis of the existing literature and empirical studies, the current standards for reliable cloud services are assessed based on recent scholarly research and developments in the technology industry.

The literature review employed a structured search strategy to examine key topics in cloud computing, cloud native technologies, learning platforms, and chaos engineering by using the following criteria:

- Only English-language publications were included
- Research articles were prioritized
- Preference was given to widely cited research articles
- Technical white papers and documentation from cloud service providers and open source tool maintainers were included for practical relevance

2.1. Key Concepts and Terminologies

Before diving into the more information-dense sections of the literature review, it is important to first clarify several key concepts that are frequently mentioned and essential for grasping the broader context of this thesis.

DevOps – A set of practices that combines software development and operations, aiming to shorten the development life cycle and deliver high-quality software continuously.

Reliability Engineering – A discipline focused on ensuring that systems perform consistently and without failure over time, emphasizing availability, fault tolerance, and resilience.

Monolith – A traditional software architecture where all components of an application are built as a single unit, often harder to scale or modify.

Microservice – A modern architecture style where an application is built as a collection of small, loosely coupled services, each responsible for a specific function, improving flexibility and scalability.

Cloud Native – Applications designed specifically to run in cloud environments, utilizing cloud infrastructure, dynamic scaling, and services to optimize performance.

Orchestration – The automated arrangement, coordination, and management of complex cloud infrastructure, typically involving managing containers or microservices at scale (e.g., Kubernetes).

Namespace – A logical partition of Kubernetes objects, that provides resource isolation in cluster level.

Chaos Engineering – The practice of intentionally injecting failures into a system to evaluate its resilience, identify weaknesses, and improve reliability of distributed systems in cloud.

2.2. Developing in the Cloud

Since the first public cloud providers offered on-demand computing, widespread Internet access has made cloud computing a key enabler of agile, fast-paced software development. Compared to on-premises hosting, where computers are hosted by the software vendors, cloud providers host a vast amount of computing power distributed across the globe, allowing customers to run and replicate their workloads on demand.

Multiple benefits come from hosting software in the cloud, such as saving on upfront capital expenses by the principles of economies of scale [1], and scaling rapidly and cost effectively based on demand or in response to fine-grained service-level metric triggers [2]. Cloud computing also splits the IT burden and associated risks between service vendors and software developers [3], which enables rapid release cycles of new solutions, by small software development teams, including startups or even by individual researchers.

Effectively, the only responsibility a software team would have, is to deliver the source code for an application, while operational expertise across all layers of the technology stack would no longer be required to release a new solution. Cloud providers typically offer a defined level of service, governed by what are known as Service Level Agreements (SLAs). These are contracts between cloud providers and customers that specify the level of service the provider commits to deliver for a given price. These agreements build trust, and violating them often results in penalties for the service provider, usually in the form of handing service credits back to the customer [4, 5].

Developing in the cloud offers developers the flexibility to build applications using a wide range of technologies and pre-existing services, tailored to their specific needs. Development teams can use multiple programming languages, embrace DevOps practices, and leverage any combination of frameworks, databases, or runtimes, while the cloud provides the underlying infrastructure to support them all. This is even more prominent in the new age of cloud native application development [6], which core characteristics and principles is to guide and ease running applications principally in the cloud. The ability to mix and match different technologies and pre-defined services in the cloud is made possible through different service models, deployment models and cloud native architecture principles, which are explored during the following sections.

2.2.1. Virtualization

Virtualization is a powerful way to decouple the hardware of a physical host computer from the application code that runs on it. Virtualization is done at different levels, most common virtualization tools like VMWare Workstation [7] and Virtual Box [8] virtualize the hardware component resources, managing simultaneously multiple different Operating Systems (OS) on top of the virtualized hardware. At the core

of virtualization is the hypervisor, which manages and allocates physical resources including CPU, memory and disk to each VM. The hypervisor ensures isolation between hosted VMs and controls how they access the underlying hardware.

Other virtualization techniques, besides hardware virtualization include, virtualization at the programming language virtualization with Java Virtual Machine (JVM), instruction set architecture with QEMU, kernel-level virtualization with Windows Subsystem for Linux version 2 (WSL) and application level virtualization with Turbo [9]. Operating system level virtualization has become increasingly popular in recent years with tools like Docker and other container orchestration tools [10] that are virtualizing a minimal operating system on top of the container runtime. This simplifies the running of different applications in isolation with minified OS to run the applications.

With virtualization, cloud providers are able to partition their existing computational resources to smaller parts that can be served on demand bases, effectively creating a repository of computational resources over the Internet ([11]). Most of the cloud resources are virtualized, including, compute resources e.g. VMs, serverless functions and raw hardware, networking [12], storage [13] and role permissions resources [14], making them accesible and easier to configure by the customers.

Virtualized workloads ease the deploying of new applications as they can be ordered with ease and on demand from cloud providers. Cloud computing benefits significantly from different inherited aspects of virtualization, particularly in its ability to scale resources based on demand. Auto-scaling is a powerful feature that promises higher availability and reliability of software when using virtualized workloads in cloud. Scaling can be either horizontal (scaling in/out), where new replicas are spun up to even the load, or vertical (scaling up/down), where allocated compute resources are switched in place for running workloads and can be triggered with different strategies [15]. Popular strategies for scaling can be resource based scaling that monitor the load on applications e.g. CPU or memory load, networking throughput based load or using predictive models to minimize cost and downtime [16].

2.2.2. Service Models

Cloud service models define how computing resources are delivered to customers, primarily based on the division of responsibilities between the cloud provider and the customer. Traditional models are typically categorized into Infrastructure as a Service (IaaS), Platform as a Service (PaaS), and Software as a Service (SaaS). These foundational models offer varying degrees of control, flexibility, and abstraction, and are often used in combination. For instance, a SaaS product may be built on a PaaS, which itself relies on IaaS. This layered interdependency is illustrated in Figure 1, which also includes a comparison with emerging models such as Artificial Intelligence as a Service (AIaaS) [17].

Service models also define the scale of how responsibilities are shared between the service provider and the customer, on each part of the cloud service. For example, in a SaaS email service, the provider is responsible of storing the emails and providing an access for each customer to read their email and send new email. Each cloud service model offers different advantages, and choosing the right one is not always

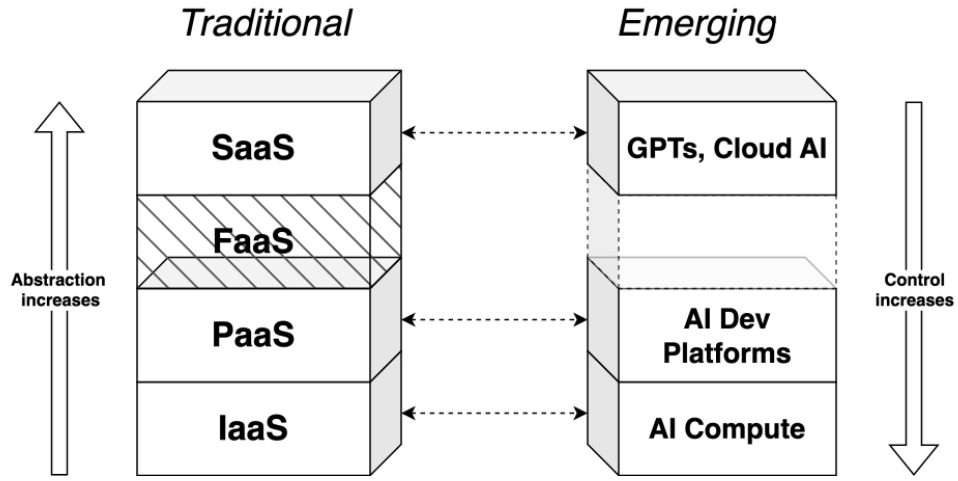


Figure 1. Layered stack diagram comparing traditional cloud service models against the emerging AIaaS model; not all layers are yet available in the AIaaS stack.

as straightforward as it seems. The best choice depends on the specific needs of the domain, as well as finding the right balance between control, responsibility, and costs [18].

In recent years, additional service models have emerged to further simplify development by abstracting more of the underlying infrastructure and operations. These emerging models—such as Functions as a Service (FaaS) [19], Database as a Service (DBaaS) [20], and AIaaS enable faster innovation by minimizing setup overhead and operational complexity. This growing trend of abstraction supports a future in which almost every part of a software system can be consumed as a service (XaaS) [21]. A more detailed comparison of traditional and emerging service models is presented in the following Table 1.

Model	Use Case	Responsibility	User Control	Pricing	Examples
IaaS	Provides virtualized computing resources.	Ensure availability of virtualized resources.	OS selection, application code, full configuration.	Pay-per-use.	Hosting VMs, storage, customizable infra [18].
PaaS	Provide platform for app development.	Manage scaling, orchestration, logging.	Application code, limited configurations.	Pay-per-use.	App hosting, Heroku [22].
SaaS	Ready-to-use application, fast delivery.	Use of the software.	No control.	Subscription-based models.	Email, Microsoft 365 Tools [23].
FaaS	Executes code in response to events.	Ensure compute runtime is available.	Runtime configuration, code.	Pay-per-invocation, execution time.	Lightweight tasks, batch jobs, AWS Lambda, Azure Functions [19, 24, 25].
DBaaS	Managed database solutions.	Backup, scaling, replication, security.	Manage schema, queries.	Pay-per-storage, compute, throughput.	AWS RDS, Azure SQL [20].
AIaaS	Cloud based AI capabilities.	Offer ML models, training infra.	Use APIs, build AI apps.	Pay-per-API or training time.	AI models & workflows, Amazon Transcribe [17].
XaaS	Aggregate of all service models.	Abstract infrastructure, platform, or apps.	Depends on the specific service.	Varies by service type.	Holistic service delivery [21].

Table 1. Overview of traditional and emerging cloud service models.

2.2.3. Deployment Models

Different deployment models exist in the cloud to address the varying needs of customers throughout the cloud development process. All models are delivered as self-service, where customers request the services and infrastructure they want to use. Cloud deployment models are typically categorized into a few different types, including private and public clouds, as well as the hybrid cloud model, which combines elements of both, but other deployment models also exist. In contrast to the on-premises model, where all infrastructure is privately owned and self-hosted in a local data center, cloud computing involves lower upfront capital expenditures and reduced maintenance costs, benefiting from economies of scale.

The on-premises model provides full control and strong security, but comes with high management responsibilities. In cloud deployment models, the cloud provider is responsible for maintaining the security and operability of the underlying infrastructure while also providing control to the customer through configuration.

Public cloud model is the most known deployment model that provides elastic workloads on-demand for its customers, customers can be individual users, collectives or businesses. Public clouds are easy to sign up for and come with small initial expenses, making them a popular choice for development and testing purposes. AWS and Microsoft Azure are some examples that provide this deployment model.

In a private cloud model, a specific cloud infrastructure is created for a customer for internal use. The customer in private cloud model is usually a business organization, and the model is often referred to as the corporate model. Private clouds are more secure than public clouds because all resources are under governance and control of the internal organization. Private cloud can be hosted on-premises by the customer themselves with full control over the infrastructure but heavy initial and ongoing expenses, a third-party vendor can host the private cloud as a managed solution that is more convenient than running on-premises but comes with extensive price. As with all virtualization, a private cloud instance can also be virtualized and deployed in isolation on normal public cloud infrastructure [26], private cloud virtualization is the most cost-effective and simplest way to establish a private cloud and is supported by all public cloud providers.

In hybrid cloud model, the customer gets essentially the best things from both public, and private clouds. The customer is usually a business organization that initially starts with a private cloud and extends it to use some integrations from a public cloud service offering. This model secures sensitive data in the private cloud, while the public cloud can be used for experimentation with development and testing. Security in hybrid cloud settings is variable, with lower security when leveraging public resources and higher protection when using private resources. When integrating public and private clouds, implementing additional layers of security is crucial due to the heightened risks of data compromise [27].

In addition, a couple of different cloud models have emerged to offer different hosting models compared to the defaults that CSPs are offering for their customers. Community cloud is a decentralized model of cloud deployment, where the infrastructure is collectively owned and hosted by the community. In this model, community members act as both service providers and customers by running workloads on a shared community infrastructure, and, in return, contribute their

underutilized computational resources to support it [28]. Other emerging practice is multi-cloud deployment model, where services from two or more cloud service providers, or from community clouds or any other cloud model, are interconnected together making a seamless grid of resources across different clouds.

2.2.4. Costs in Cloud

Cloud is characterized as a cost-effective way to deliver distributed applications across the globe. By the economics of scale [1], the costs of running workloads in the cloud will decrease as CSPs invest in larger data centers. As Gartner [29] forecasts that public cloud computing popularity and expenditure will increase by 21 in 2025, and CSP providers like Microsoft are actively investing in new cloud regions [30], it can be logically estimated that cloud computing prices will decrease over time. However, automated service provisioning through auto-scaling and other forms of automation can lead to unexpected costs in the cloud. In some cases, unnecessarily excessive resources may be allocated to idle workloads with no tasks to process, resulting in avoidable expenses. These issues can significantly elevate operational costs in the cloud but can be mitigated by monitoring resource utilization and tracking vertical and horizontal auto-scaling events to identify inefficiencies.

In cloud, customers can choose the hardware for their workloads, with more powerful virtual machines incurring higher costs. Right-sizing resources is an effective way to manage expenses, as vertically scaling compute resources is relatively easy in cloud environments. Moreover starting with small VMs and horizontally scaling on-demand is a great strategy to mitigate costs as horizontal scaling causes no downtime in the service [15].

Containers should be used where possible, as they are lightweight virtualizations of the operating system and application. This allows more applications to fit on a single physical host compared to virtual machines. Tools like Kondense [31] exist to further optimize the container orchestration of Kubernetes by continuously measuring the optimal amount of resources needed by workloads and continuously vertically scaling them. SHOWAR [32] is another auto optimizer that scales small workloads, both horizontally and vertically to find the optimal resource balance, it was shown to improve the resource efficiency by up to 22%. Also, FaaS services are the best solution to cut down on infrastructure costs for short-lived batch jobs and other background processes

Most CSPs offer built-in tools and services, such as AWS Cost Explorer [33], and Azure Advisor [34] to help monitor, analyze, and optimize resource usage and cost efficiency. These tools are used to ensure effective cloud cost management by detecting underutilized resources, recommending optimal instance types, and providing automated cost-saving suggestions. They support budget predictability and financial planning, and assist in aligning resource usage with actual and forecasted demand ,and reduce unnecessary spending [35]. In addition to vendor-specific options, many popular open source tools like Prometheus [36] can be deployed with the application workloads, to collect and visualize application metrics. Regardless of the monitoring stack used, it is a best practice to continuously observe resource utilization to identify idle or over-utilized workloads and take timely action.

2.2.5. Cloud Native

Cloud developing can be used to build highly scalable, resilient applications, where new features can be delivered frequently in small release cycles. Cloud native is the most recent trend to approach the management of modern applications in cloud environments. Cloud native applications are distributed, elastic, and horizontally scalable systems, Built using loosely coupled microservices with only a minimal number of stateful components [37]. The Twelve-Factor App [38] is a methodology that heavily supports the cloud native development approach for delivering modern, scalable, and maintainable applications in the cloud.

To support and standardize this approach, the Cloud Native Computing Foundation (CNCF) [39] was established by the Linux Foundation as a non-profit to govern a ecosystem of the fastest growing, vendor-neutral, open source projects that form the backbone of modern technology infrastructure in the cloud. CNCF drives community collaboration through maintaining projects and best practices, and through community meetings, like Kubernetes Community Days that are organized across the globe [40] to accelerate the innovation in cloud native ecosystem.

There is growing industry adoption of cloud native tools and consensus on what constitutes to the paradigm, and academic research should catch up with the industry to address the long-term impacts and engineering complexity introduced by new cloud native development paradigms [6]. Cloud native represents a future-proof strategy for providing cloud services, as it enhances portability, scalability, resiliency, and developer experience while ensuring cloud-readiness. These factors are crucial for building successful applications in dynamic, distributed environments [38].

2.3. Cloud Native Architectures

Cloud Native Architectures are foundational to modern application development, they emphasize scalability, modularity, and resilience within cloud environments, and mark a distinct shift from monolithic application architectures to more distributed software systems, where applications are decomposed into loosely coupled components that can be independently developed and deployed. These architectures are specifically designed to utilize the elasticity of the cloud, enabling systems to adapt under varying demand, recover from failures gracefully, and evolve over time without major rework. As organizations increasingly migrate workloads to cloud platforms, cloud native design principles have become essential for building robust, flexible, and cloud-optimized applications.

2.3.1. Microservices

A microservice is a workload running in isolation in cloud, designed as a distinct unit to do a specific task, and to do it well. Microservice architectures (MSA) are built with distributed, loosely coupled microservices as building blocks, to deliver entire applications that are distributed by their nature, and can be developed in isolation. Microservices are self-contained units that encapsulate all necessary

dependencies to perform their designated functions [41]. This architectural approach enhances maintainability, as smaller, focused services are easier to develop, test, and independently deploy to production environments. Consequently, microservices simplify the overall software development lifecycle.

The adoption of microservice architecture has led to substantial improvements in system automation and reliability. By organizing functionality into loosely coupled, independently deployable services, teams can automate rapid development cycles and push updates to production with high frequency. This approach enables deployment rates to scale from occasional releases to hundreds per week [42]. Without compromising the reliability, but instead increasing it with minimization of cascading failures.Furthermore, scalability is inherent to microservice architectures, as they are designed to be elastic and can scale both horizontally and vertically.

Microservice architectures have been driving to become the industry standard, while major technology companies like Netflix, Amazon and ThoughtWorks have pioneered these methodologies for over two decades, and influenced the architectural standards with real-world implementations [43]. The services offered by cloud based companies are typically designed according to microservice principles, making them inherently modular and independently scalable.

Containers are highly effective in building microservice architectures due to their lightweight, self-contained nature, and ease of deployment and portability. They provide isolated application environments that can run consistently, across different machines equipped with a compatible container runtime. Docker, the first and most widely adopted container platform, played a pivotal role in popularizing containerization in cloud native development. Docker utilizes containerd as its default container runtime, alternative runtimes such as CRI-O have emerged and, in several benchmark studies, demonstrated better performance compared to containerd [44]. Although Linux containers were the original implementation, support has since expanded to include architectures for Windows containers and Apple Darwin-based systems. For broader cross-platform compatibility, QEMU virtualization is used to build containers for any target architecture regardless of the host system [45]. The CNCF explicitly promotes microservices as a foundational architectural style for cloud native applications. Its reference architectures are all designed around containerized deployment models.

One major area of concern involves dependency management. As microservices are typically small and provide just enough code to function, they rely on numerous external libraries and packages, and as these dependencies evolve, they may introduce breaking changes, bugs, or even security vulnerabilities. In addition, microservice architectures are also harder to monitor. Unlike monolithic systems, microservices are inherently distributed, often involving many independently deployed services. This complexity makes it difficult to track overall system health and performance and demand a distributed holistic monitoring approach. This includes the adopting service logging, metrics collection, and distributed tracing in early phases of development. These practices help teams detect issues quickly, understand service-to-service interactions, and maintain high product quality and system reliability [46].

2.3.2. Container Orchestrators

Managing microservices at scale requires a container orchestration platform to automate deployment, scaling, and operation of containers across clusters. While tools like Docker Compose are useful for local development and single-host environments, they are not suitable for managing distributed workloads across multiple nodes [47]. To handle the complexity of modern cloud native systems, more advanced orchestration tools are necessary. These platforms provide cluster scheduling, state management, high availability, service discovery, and support for automated deployments [48].

Among these, Kubernetes (K8s) is the most widely adopted and feature-rich orchestration platform, often used as a reference architecture by the CNCF and highlighted in the CNCF Landscape among over 40 orchestration tools available today [49, 50, 51]. Alternatives such as Docker Swarm and Apache Mesos offer simpler or more specialized approaches. For instance, Docker Swarm is available natively with Docker and is easier to set up, while Mesos is designed for large-scale distributed workloads and emphasizes security and scalability [50]. A more comprehensive comparison of these and other container orchestration tools is presented in Table 2.

While Kubernetes offers unmatched flexibility and control, it requires significant technical expertise and operational effort to manage. CSP-managed alternatives such as Amazon Elastic Kubernetes Service (EKS) and Azure Kubernetes Service (AKS) [52, 53] handle the control plane on behalf of users, thereby reducing operational complexity. Additionally, control plane–free services like AWS Elastic Container Service (ECS) and Azure Container Instances (ACI) [54, 55] provide simplified deployment options that still offer Kubernetes-level availability and scalability. These solutions serve as more accessible entry points, depending on needs. Ultimately, the choice of orchestration platform involves trade-offs between scalability, complexity, and flexibility.

Tool	CNCF Status	Scalability	Complexity	Control Plane	Managed by CSP	Notable Features
Kubernetes	Graduated [51]	High	High	Yes	Yes [52, 53]	Most widely adopted; robust feature set; strong ecosystem [49, 50], can also be run on bare metal [56].
Docker Swarm	–	Medium	Low	Yes	No	Native to Docker; simple setup; less scalable [47, 50].
Apache Mesos	–	High	Medium	Yes	No	Highly scalable; secure; used in large-scale clusters [50].
OpenShift	–	High	High	Yes	Yes (Fully-managed by Red Hat)	Enterprise-ready; Kubernetes-compatible with built-in continuous integration.
ECS / ACI	–	Medium	Low	No	Yes [54, 55]	Simplified deployment; limited customization and extensibility.
Minikube	–	Low	Low	Local-only	No	Lightweight K8s for development and testing [57].

Table 2. Feature comparison of common container orchestration platforms.

2.3.3. Version Control Systems

Distributed Version Control Systems (DVCS) have been adopted by many organizations to enable efficient teamwork against codebases to have multiple developers work on the same application concurrently. At simplest, version control involves keeping multiple copies of an application at different stages and labeling them to indicate their version [58]. This approach, often referred to as manual version control, relies on duplicating files and quickly becomes very complex and error-prone. In centralized version control systems (CVCS) there exists only one central repository for the application code, this was inherently better than the manual version control systems, but it introduces several risks. If the connection to the server hosting central repository fails, access to the full project history and collaboration capabilities is lost, also developers were able to access only the latest version of the code locally, limiting offline and flexibility [59].

In Distributed Version Control Systems (DVCS), every user has a full copy of the repository stored on their local machine, all remote repositories are based on the same root commit. This setup enhances collaboration, improves code quality, and has become a preferred solution regardless of team size and with geographically dispersed teams [60]. DVCSs have been successfully adopted by many open source software projects, resulting in increased contributor participation, improved traceability of contributions, and a clearer understanding of community growth and development dynamics [61].

Git is by far the most popular DVCS when measured by searches and questions posted in Stack Overflow [58]. It was originally developed to aid the collaborative development of the Linux kernel, and is now adopted by many companies as their version control system. Git [1] is a powerful command line tool, that is used to keep track of different versions of files in distributed manner. Git actions include, but are not limited to, staging files, committing to record changes, revert to undo specific commits, reset to move the repository to a previous state, and push and pull to synchronize changes between remote repositories. As a fully functional command-line tool, it serves as an effective solution for use in scripting and automation tasks.

cloud based Source Code Management (SCM) platforms, such as Github, extend the capabilities of traditional VCSs acting as centralized repositories and adding collaboration features that enhance team productivity and communication. Some features include multi repository support, access management, code review support before merges, integration to build, automated testing and deploying of code and vulnerability scanning and analysis [62]. These features increase the reliability of development by enabling automated testing and code quality scans from the SCM.

To align with the cloud native practices, the Twelve-factor methodology [38] emphasizes that each application should have a single codebase managed through a VCS. This codebase acts as the single source of truth. Applications should not be split across multiple repositories, nor should multiple services be bundled into a single monolithic codebase. Additionally, the methodology emphasizes that all dependencies must be explicitly declared in a dependency manifest and committed to the repository. Any tools or libraries needed to run the application should be vendored

[1] https://git-scm.com/book/en/v2

into the codebase to ensure consistency and portability across different deployment environments.

2.3.4. GitOps

GitOps [63] is a set of practices to enable development teams to achieve continuous deployment in cloud native application development. GitOps enables development teams to achieve continuous deployment by using VCSs like Git to store all necessary configurations. The core idea of GitOps is to have the repository to be single source of truth and define the desired state of application declaratively with manifest files in the repository. Declarative infrastructure code results in more reliable and readable code, by focusing on the desired end state of the system, rather than the steps to achieve it.

GitOps brings multiple benefits, all changes to the code happen in the version control system, and no additional tools are needed for operations, Treating operations as code enhances the developer experience and results in better productivity as developers are using familiar tools for both development and operations. All benefits that were found with SCMs during Version Control Systems in 2.3.3 apply for GitOps, but it also allows for more frequent deployments, easy rollbacks with Git reverts, and version tracking including complete history of every change ever made to the system [63].

GitOps can be implemented using two primary operational models: push-based and pull-based. In the push-based model, the continuous integration pipeline builds software artifacts when new code is pushed to the production branch in version control system. The continuous delivery process then deploys these artifacts to the desired environment. In contrast, the pull-based model relies on an operator that continuously monitors the desired state stored in version control and compares it to the actual state reported by the cluster. When differences are detected, the operator updates the cluster to reflect the desired state.

Two GitOps tools under the CNCF have attained the highest graduated project status, signifying mature governance, broad production use, robust community support, and long-term sustainability commitments [51]. ArgoCD and Flux both operate purely with pull-based workflows and automate and reconcile container workloads to Kubernetes clusters according to specified declaration manifests. Helm is another CNCF project that has achieved graduated status. It streamlines Kubernetes deployments through the use of parameterized templates that are referred to as Charts. These charts contain parametrized Kubernetes manifest files that can be deployed through automation with ArgoCD or Flux. It must be mentioned that while most GitOps operators are implemented with Kubernetes clusters [64, 63], GitOps practices are not limited to only Kubernetes and can be used with other available IaC tools.

2.3.5. Monitoring and Observability

Monitoring in the cloud involves continuously keeping tracking and analyzing the performance, health, and availability of cloud based applications and infrastructure. By analyzing metrics and logs with monitoring tools, it provides real-time insights across various cloud resources, offering actionable data from infrastructure and applications.

Due to the increase in popularity of microservices and containerized applications, monitoring of distributed systems have evolved over the years to be more complex with increasing amounts of decoupled dependencies [65]. In microservice system processes are spread across many hosts machines and monitoring them is a challenging task and requires distributed monitoring systems that can collect, distribute and process the information simultaneously from multiple different microservice components [66]. Observability is becoming an essential measure in microservice architectures of how well a system's internal state can be inferred from its external outputs. It reflects the extent to which infrastructure and application interactions can be effectively monitored and understood. Building observable systems enables end-to-end visibility into runtime behavior, allowing for the rapid detection of anomalies, identification of issues, and mitigation of potential degradations in service reliability [67].

Several strategies are required to enable holistic microservice monitoring and observability, and include the generation, storage, processing, and presentation of monitoring data [68]. Monitoring data must be generated across all layers of the system, involving the systematic collection of runtime and interaction-level information at the host, platform, and service levels. Instrumentation is enabled through agents that are installed on the host machine and continuously measure the performance of the host, with containerized architectures, a sidecar model has become popular where a separate container agent is deployed in parallel with the application to gather, process and deliver metrics to other. To ensure historical visibility and enable deeper analysis, centralized storage is recommended to retain metrics over time. This facilitates the processing of raw data into meaningful time series aggregations, helping to uncover long-term trends and detect anomalies. The presentation of monitoring data provides users with a unified view of key metrics across the distributed microservices architecture.

The most commonly monitored metric in microservice systems is resource usage, such as CPU and memory consumption, followed by load balancing and availability [69]. These metrics have proven useful for a variety of purposes, including enhancing customer experience by preventing failures and improving performance, and evaluating application behavior under high traffic. They also support monitoring the overall health of host infrastructure, tracking the number of active hosts and pods, proactively identifying potential failure points, and maintaining system availability.

Logging is widely used to assess the state of systems and can be used to do something when the log entries represent specific request, have timestamp, include response status and code, and include unique id for reference.

Distributed tracing revolves around the concept of a trace, which captures the complete journey of a request as it flows through an application. A trace is composed of multiple spans, each representing a discrete unit of work performed by an individual service. These spans include timestamp data and may reference one another to form client-server relationships, creating a logical chain of processes across services [70]. Each span can be thought of as marking the start and end of a specific operation within the process chain. Additionally, many asynchronous operations may execute concurrently, resulting in spans that overlap in time, reflecting the parallel nature of distributed systems.

The study of cloud native monitoring tools should begin with an investigation of the OpenTelemetry (OTel) framework [71], which has recently gained significant

industry attention and is emerging as the de facto standard. OpenTelemetry is a vendor-neutral cloud native framework that unifies the collection of metrics, traces, and logs in distributed systems. Its standardized approach simplifies observability across diverse services and environments. Backed by the CNCF and a strong open source community, OpenTelemetry has seen widespread adoption, as evidenced by its support across monitoring tools and cloud service providers offering integrations with other observability platforms.

To export telemetry data to other observability platforms, an OpenTelemetry Collector is required. The collector runs as a separate container in the infrastructure and listens for telemetry from various sources. These sources may include application endpoints that expose telemetry, Prometheus [36], or other cloud native monitoring tools. The collector ingests and processes the data using a pipeline model, where different processors can be applied to transform, filter, or aggregate the telemetry. The choice of processors depends on the specific version and distribution of the collector.

Many cloud providers and vendors already offer their own flavor of the OpenTelemetry Collector and have contributed their own exporters to the SDKs. These solutions often include additional features to allow easier integration with cloud environments. For example, Microsoft Azure Monitor OpenTelemetry Distro [72] is a fork of the upstream community collector, that supports more seamless integration with other monitoring services in Azure.

There are numerous open source Application Performance Monitoring (APM) tools that enable monitoring and observability in microservice architectures at scale. CNCF supported projects include Prometheus [36] (for metrics collection and visualization), Jaeger [73] (for distributed tracing and visualization), and FluentBit [74] (for log forwarding and processing) are widely adopted throughout the industry. These tools offer scalable and flexible solutions for collecting and presenting telemetry data, including logs, metrics, and traces, in distributed systems. Typically deployed as containers alongside application workloads, they support the operation of self-managed observability stacks in both local and production environments. Another CNCF project, Grafana [75] , is widely used to visualize metrics through customizable dashboards and can serve as a central component of a powerful APM platform when combined with other CNCF observability tools.

Other open source tools have been developed to extend Prometheus with features for scalability and long-term storage. Cortex [76] enables high availability multi-tenant metrics storage for Prometheus and supports various backends, including self-hosted deployments or with cloud service providers. Thanos [77] extends Prometheus by adding global query capabilities, deduplication, and long-term metrics storage, making it suitable for large-scale distributed services with historical observability needs.

Kubernetes monitoring tools have emerged to improve real-time observability and incident response. Anteon [78] provides visibility into Kubernetes environments by providing real-time dashboards and simplified observability workflows for developers. BotKube [79] acts as a Kubernetes monitoring and automation tool that integrates with communication platforms, allowing teams to receive alerts and monitor cluster changes directly from their chat tools.

Several commercial observability platforms, members of CNCF are listed in the landscape. Azure Monitor is Microsoft Azure [80] native observability platform that supports comprehensive monitoring of metrics, logs, and traces, with integrated

application insights for performance management. Datadog [81] provides a cloud-hosted, all-in-one observability service supporting infrastructure, applications, and user monitoring. It offers support for telemetry data and includes real user monitoring as a core feature. Dynatrace [82] provides an AI-powered observability platform that includes infrastructure monitoring, APM, and continuous automation for cloud native environments. The CNCF Landscape includes over 140 monitoring and observability tools, making it impractical to analyze each in detail. To narrow the scope, Table 3 compares a selection of the most widely adopted tools within the CNCF ecosystem, along with a few others that are particularly relevant to the focus of this thesis.

Tool	CNCF Maturity	Focus	Pull-based	Push-based	Storage	OTel Support	License	Cite
Prometheus	Graduated	Metrics	✔	✗	✔	✔	Open source	[36]
Jaeger	Graduated	Traces	✗	✔	✔	✔	Open source	[73]
Fluentbit	Graduated	Logs	✗	✔	✗	✔	Open source	[83]
Grafana	Sandbox	Visualize	✔	✗	✔	✔	OS/Commercial	[75]
Cortex	Incubating	Metrics	✔	✗	✔	✔	Open source	[76]
Botkube	Member [a]	Visualize	✔	✗	✗	✗	Open source	[79]
Anteon	— [b]	Visualize	✔	✗	✗	✗	Open source	[78]
OpenTelemetry	Incbuating	Telemetry	✗	✔	✗	✔	Open source	[71]
Thanos	Incubating	Metrics	✔	✗	✔	✔	Open source	[77]
Azure Monitor	Member [a]	Visualize	✗	✔	✔	✔	Commercial	[80]
Datadog	Member [a]	Visualize	✗	✔	✔	✔	Commercial	[81]
Dynatrace	Member	Visualize	✗	✔	✗	✗	Commercial	[82]

Table 3. This table lists the potential failure modes across all micro-services in the reference application. The modes are ranked from highest to
lowest severity as perceived by the user.

[a] The tool is maintained by an organization officially affiliated with the CNCF.
[b] Listed in the CNCF Landscape but not a CNCF-hosted project.

2.4. Self-Service Learning Platforms

Hands-on learning is essential for developing practical skills that can be directly applied in real-world work environments. Contained laboratory environments can be rapidly provisioned using on-demand virtualization and should be accessible through self-service mechanisms to maximize efficiency and enhance the developer experience. In educational contexts, self-service hands-on, have been shown to foster collaboration, deepen student understanding, and improve academic performance [84].

In this section, various approaches for delivering hands-on cloud laboratories through self-service learning platforms are studied. Particular emphasis is placed on the types of service catalogs offered by these services, technologies supported, especially in relation to their support for cloud native development. Usability of these learning platforms is interesting part, how are the laboratories provisioned to the students, how are they accessed and the degree of autonomy granted for students during the learning process. In addition, associated cost models and licensing fees are searched to analyze the pricing trends of cloud based learning platforms.

2.4.1. Lab-As-A-Service Model

Azure Lab Services Platform is a managed service provided by Microsoft that enables organizations to order scalable, pre-configured virtual learning laboratories within Azure Portal as a service. Labs-as-a-Service (LaaS) model facilitates on-demand provisioning of laboratory environments, all of which are managed through automated IaC templates behind the scenes [85]. The use of Lab Services requires at least one Azure subscription under which the laboratories are hosted. All participants, including educators, administrators, and students, would need access to the subscription, and sufficient role-based access control (RBAC) permission to manage laboratory resources.

The platform includes an offering catalog, but instead educators are required to design laboratory exercises using individual laboratory plans. These lab plans are relatively simple and primarily require information about the virtual machine image used, access policies, and lab lifecycle management details, such as when the lab will be provisioned, and when it will be decommissioned. To put it simply, the lab plan spins up a virtual machine with specified access control assigned to specific students to access that virtual machine. The pricing model is on-demand, and costs are incurred only for the time that the lab virtual machines are running.

Unfortunately, Microsoft already announced the retirement of Azure Lab Services by June 2027. At that point, all existing laboratory plans will be removed, and the platform will transition towards the use of alternative partner solutions [85].

Other service providers like GoDeploy [2] provide commercial LaaS platforms for its customers to be used in laboratory based training. It enables cloud based deployment of fully configured lab environments through the cloud infrastructure, allowing educators to design hands-on laboratory trainings without setting up any own infrastructure.

[2] https://godeploy.com/

2.4.2. Cloud Native Learning Platforms

Labs4grabs [86] was a learning platform for Kubernetes, that included ephemeral hands-on laboratories, provisioned from a web user interface. The laboratories were deployed to the cloud infrastructure managed by the maintainers of the project, and users would pull the necessary configurations on their local machines, to connect to their laboratories in the cloud. Each lab comes with a step-by-step guide, and the lifecycle of laboratories was managed with automation. However, the project has already been discontinued by the maintainers due to rising infrastructure costs.

Katacoda [87] was another Kubernetes learning platform maintained by O'Reilly. Themes included supported learning software engineering through hosted interactive Kubernetes platform. Unfortunately this platform has also shutdown and is no longer in production.

Collabnix Kubelabs [88] is a learning platform for cloud native technologies, like Docker and Kubernetes, that shows active community engagement, and lists multiple maintainers for the project. Laboratory themes include learning the use of control plane services, Kubernetes networking, GitOps practices, how to monitor Kubernetes and deep dives to certain topics. The platform does not support Kubernetes platform for running the resources, instead Kubernetes cluster needs to be hosted by the students and Kubelabs provides step-by-step guides and resource manifestations to go through learning labs. Kubelabs provided learning platform is free of charge, and good way to learn Kubernetes with hands-on laboratories.

KodeKloud [89] is a commercial learning platform for cloud native technologies, like Kubernetes, Docker and Linux, and supports multiple different learning paths for cloud native technologies with technology and role-based focuses. Themes include learning developer, administrator and multiple engineering level skills for cloud native technologies. Learning paths are customizable and contain hundreds of hours of learning material. KodeKloud provides the platform for running the laboratories and includes a interactive shell in browser to progress the laboratories. The platform supports Site Reliability Engineering (SRE) learning path, but it does not cover any chaos engineering practices. The platform is commercial and learning paths require specific licenses to be used. Study plans for business organizations start from 330 $ / user / year.

Most cloud native learning platforms focus exclusively on Kubernetes. Some platforms have offered fully hosted laboratory environments on their own infrastructure. However, many of these have since shut down due to rising infrastructure costs and a lack of sustainable financial support. Other hands-on Kubernetes platforms provide only step-by-step learning guides, requiring users to host their own infrastructure in order to gain practical experience. Additionally, a few commercial platforms offer end-to-end capabilities by hosting the labs on their own infrastructure and providing in-browser interactive shells, but these solutions typically come at a high cost.

Building on the practices used by existing platforms to provide hands-on learning, similar principles are incorporated into the learning platform developed in this thesis. The aim is for automated infrastructure provisioning, interactive lab environments, and cost-effective design to be combined in a way that ensures sustainability, accessibility, and a practical learning experience for students.

2.4.3. Spotify Backstage

Backstage is an IDP framework developed by Spotify to address the growing complexity of managing microservices, infrastructure components, and developer tooling at scale. After its initial release, Backstage was open source licensed and donated to the CNCF, and is currently an incubating project maintained by Spotify. Backstage enables the creation of self-service workflows that empower developers to manage their own services and reduce operational dependency on platform teams. It aligns well with the principles of cloud native architecture by promoting automation, declarative configurations, and service discovery through its service catalog. Several core features of Backstage will be introduced, focusing on their potential to facilitate the development of self-service learning platforms.

One of the core features of Backstage, is building automated workflows with the scaffolder plugin [90], which allows platform teams to build reusable and scalable workflows for any repetitive engineering tasks. Running software templates reduces setup time, minimizes manual interventions and ensures reliable, repeatable best practices across teams.

Software templates in Backstage typically consists of an input stage, a sequence of automated actions, and an output stage. The template inputs may include text fields, booleans, arrays, or various picker components. Additionally, custom input fields can be defined and integrated into the template to accommodate specific use cases.

Core Backstage includes a set of built-in actions that can be used to perform common tasks such as fetching remote content and publishing new Git repositories. To support more specialized workflows, custom actions can be developed and integrated into software templates. These actions are executed in a defined sequence using the computational resources provided by the running Backstage instance. The output of a template is typically rendered as a markdown file, which displays the results of executed actions or presents other relevant information to the user.

The software catalog [91] provides a centralized inventory of various entities, primarily software components and systems, which organizations can aggregate from otherwise loosely coupled sources into a unified service catalog. Software components can be registered manually through the catalog user-interface or with automated custom entity providers that ingest metadata from external systems such as version control systems. Catalog entities can be further annotated with key-value metadata, this metadata is used to support different functionalities of catalog and plugins.

In addition to technical components, the software catalog supports other organizational metadata with entity kinds like User and Group. These entities reflect individuals and teams within an organization and can refer to each other through relations. For example, a User entity can include a relation indicating group membership, while a Group entity can define its members explicitly. Catalog entities can be further annotated with different key-value pairs to enrich the metadata context for improved discovery and filtering.

Backstage features a flexible authentication system to identify users within the application [92]. This system plays a key role in managing user access and integrating organizational identity. In addition to authentication, Backstage provides a RBAC framework that enables fine-grained permission management across plugins, software templates, and catalog resources for users and groups.

Finally, Backstage plugin ecosystem is extensive and continues to grow. A wide range of cloud native plugins are made available by Backstage, including the Kubernetes plugin, which aggregates information from annotated workloads and provides a unified view of the service it within the catalog. Additionally, new template actions can be introduced with scaffolder plugins, extending the capabilities and use cases of software templates. Backstage community [93] has contributed open source plugins that can be freely integrated into the platform where appropriate. In addition, some commercial plugins exist but are behind paywall.

2.5. Chaos Engineering

With DevOps practices, it has become common to thoroughly test application code and establish sufficient confidence before deploying new features. Over time, even the most reliable code will encounter failures, as nothing can function flawlessly forever. In distributed software systems, there are too many potential failure scenarios to cover, including some beyond the control of developers or REs. Once an application is deployed in production, it can encounter many uncertainties, such as unstable networks, faulty disks, throttling CPUs, and more.

Chaos Engineering is a discipline that assesses the resilience of systems in a production environment [94]. While a well-designed distributed system should handle unanticipated conditions effectively, a poorly designed system may become unusable, leading to significant drawbacks for development teams and for the business. Unlike traditional unit testing, which primarily focuses on application-level failures, chaos engineering also addresses failures outside the application scope, such as those occurring in infrastructure and networks. Originally introduced by Netflix [95], chaos engineering is now a well-recognized practice with extensive tooling and native support from cloud providers [96].

Chaos engineering is built upon a set of core principles that guide its practices and methodologies [94]. First, a hypothesis is built based on the steady state of a distributed system, which includes measurable metrics such as RAM usage, CPU usage, and others that are continuously monitored. Next, the chaos experiment should replicate a real-world scenario; for instance, it could simulate a cloud provider's data center blackout. These chaos engineering practices should be executed in the production environment to build confidence in the systems being operated, although a digital twin or a staging area, that closely resembles the production environment, could also be used [97]. Since it is impossible to predict all potential failures in the production system, continuous testing with varied chaos experiments is a recommended practice. These principles are crucial for ensuring that chaos engineering experiments are conducted effectively and safely, helping the practice meet its primary objectives, and are typically performed during a designated chaos game day.

Multiple tools are available for conducting chaos engineering in distributed systems. Netflix has released its solution, Chaos Monkey, as open source, and several other open source tools also exist. Additionally, cloud providers are beginning to develop native tools for running chaos experiments against their platforms, such as Azure Chaos Studio (ACS) [96]. However, some tools are vendor-locked, such as Chaos Monkey with AWS, and this should be considered when selecting chaos engineering tools.

Chaos experiments offer numerous benefits, such as increased system resiliency, improved incident response, and enhanced confidence among development teams in their software systems and infrastructure providers' capabilities. However, implementing chaos engineering is challenging and requires substantial effort to persuade developers and stakeholders to trust the process of building a more resilient system. Despite understanding the benefits, the uncertainty and potential costs associated with chaos experiments can be daunting. Therefore, it is crucial to make chaos engineering tools easily accessible for REs to facilitate testing and learning, helping to overcome these challenges and fully realize the advantages of chaos engineering.

2.5.1. Software Risk Assessment

Software Risk Assessment is the process of identifying, analyzing, and prioritizing potential risks that could impact the reliability and performance of an application. In complex and distributed systems, this assessment is especially critical, as it helps proactively uncover vulnerabilities and weak points that could lead to service disruptions or degrade the user experience.

Failure Mode and Effects Analysis (FMEA) is a systematic problem solving approach for complex systems to assess their reliability analysis. FMEA started in experimental military applications and has since been adopted by the software industry [98]. The primary objective of FMEA is to mitigate risks by identifying potential failure modes within a system, process, design, or service and evaluating the consequences of these failures. This is achieved by assigning a Risk Priority Number (RPN) to each identified failure mode, which guides the prioritization of corrective actions. FMEA has been widely adopted due to its effectiveness in improving reliability, safety, and quality [74].

For distributed services, developers typically address application-specific issues through unit and integration testing. Although these tests are effective in verifying functionality and interactions within and between services, they often overlook broader failure scenarios related to the underlying platform, infrastructure, and network. FMEA complements traditional testing by identifying these additional risks, such as service outages, network latency, or dependency failures. Once these failure modes are identified and prioritized, they can be used to design targeted Chaos Engineering experiments, allowing teams to proactively test system resilience under real-world conditions.

Root Cause Analysis (RCA) is a systematic approach that is used to identify and address project issues, aiming to prevent the problems reoccurring in future. These issues can include schedule delays, cost overruns, and software performance problems. RCA involves collecting data, doing analysis on the data, and finding root causes using various methods, to improve outcomes and customer satisfaction [99]. In software development, RCA helps understand past failures and ongoing issues, reducing the likelihood of future software failures. While RCA is traditionally conducted in face-to-face meetings, RCA faces challenges in distributed teams due to the lack of real-time tool support, making synchronous retrospectives difficult [100]. RCA methods

and processes are crucial for Reliability Engineering in distributed software projects, as they focus on improving software reliability and reducing the likelihood of failures.

Distributed tracing can be helpful tool during RCA, by identifying the initiator of a sequence of service calls and capturing the direction and frequency of those interactions, it becomes possible to isolate the exact service responsible for performance degradation or failure. When aggregated over time, distributed traces reveal patterns in service dependencies and call frequencies at both the instance and service-type levels [68].

2.5.2. Reliability Metrics

To meet the requirements for running Chaos Engineering, a holistic monitoring approach must be established. However, monitoring by itself is not enough and good metrics that can define the state of the system need to be identified. Therefore, it is essential to define a focused and meaningful set of metrics that accurately reflect the reliability of the system and guide fault injection and resilience analysis effectively.

To understand the user experience of an application, it is essential to monitor critical metrics that reflect system performance and reliability from their perspective. Service-Level Indicators (SLIs) are carefully defined quantitative measures of the level of service provided. Common SLIs include request latency (response time), error rates, and availability [101]. These metrics are particularly important in the context of Chaos Engineering, where they serve as benchmarks for detecting system degradation during fault injection experiments.

SLIs are typically aggregated over defined time windows and presented as rates, averages, or percentiles to better capture user-facing service quality. Non-functional requirements (NFRs) define the technical expectations and quality standards for distributed services. Chaos Engineering helps validate whether these NFRs hold under adverse conditions by deliberately injecting faults and observing system behavior in real-time.

Unlike functional requirements, which are usually subject to rigorous testing during development, NFRs are often treated as lower priority and tested in an ad hoc manner [102], or not tested at all. This trade-off, made in favor of rapid development, can lead to undetected performance and reliability issues. Chaos engineering complements traditional testing by actively validating NFRs under stress, making it crucial to evaluate these requirements from the user's perspective. The metrics used should not only be technically accurate but also serve as meaningful indicators of real-world user experience, especially during unexpected system failures.

2.6. Conclusion and Research Gaps

Recent research highlights the increasing adoption of cloud native tooling within modern cloud computing environments. Core practices such as microservices, container orchestration with Kubernetes, GitOps workflows, and observability frameworks are now central to the design of scalable and resilient systems. These

technologies enable automated infrastructure management, improve deployment speed, and provide better reliability.

In parallel, self-service learning platforms have gained traction as a means of delivering hands-on technical education. Lab-as-a-Service models and internal developer platforms like Backstage showcase how automation and declarative workflows can be utilized in provisioning and managing learning environments, making them more efficient and accessible.

A consistent pattern across the reviewed platforms is their reliance on automation to reduce manual intervention and improve scalability. However, many existing implementations face notable limitations, including high operational costs, limited customization options, or the absence of dedicated learning paths on specific topics. Open source platforms often require learners to host and manage infrastructure independently, creating a barrier to entry, while commercial offerings tend to limit flexibility through proprietary constraints. In addition, developer platforms with automation capabilities, such as Backstage, remain largely unexplored in academic research.

This study addresses this gap by exploring how different cloud native tools can be applied to build an automated, cost-effective learning platform with self-service capabilities in the cloud.

3. METHODOLOGY

In the methodology section, the future technological landscape of the learning platform will be explored and multiple potential solutions will be evaluated, weighing their suitability based on predefined criteria. Various solutions addressing questions will be introduced, such as: where the laboratories will reside, who will have access to them, how that access will be managed, how the laboratory life cycle will be handled, which software solutions will be used, what are the potential failure modes, the chaos engineering frameworks employed to evaluate these failure modes, and how the application's health will be monitored. My goal is to identify tailored solutions from the proposed options, all of which adhere to cloud native practices. Additionally, each question addressed in this chapter comes with a set of requirements and prerequisites, which will be presented before discussing the solutions.

3.1. Requirements

The requirements are set as early as possible to ensure that the implementation of the learning platform is guided in the right direction. The setting of these requirements is intended to help avoid common pitfalls during development, including solutions that may conflict with cloud native best practices or hinder the progress of this study. Additionally, the number of potential solutions discovered during the exploration of different approaches is deliberately narrowed. The requirements related to laboratory execution, provisioning, software needs, risks, chaos engineering, and monitoring solutions will be presented in the following sections.

3.1.1. Running the Platform

The chosen cloud provider for the platform is Microsoft Azure, this is given as OP's strategy is to run its operations on top of Azure in future. No other cloud providers are evaluated during this study. However, since cloud native technologies are designed to be cloud agnostic, the platform could also be deployed to respective services in other cloud provider environments. Multi-tenancy or cross-cloud implementations require more complex configuration and cannot be achieved out of the box with the current setup, but they could improve the platform's scalability. Kubernetes and its many flavors are considered for the platform due to their flexibility, and strong support from cloud providers, as summarized in Table 2. During the first laboratory exercise, a reference application will be deployed to provide a real system for conducting chaos experiments against. Backstage will be used as a central hub for authorizing our users and for facilitating the ordering process of lab environments, through customized software templates.

The lab engine will logically partition each laboratory environment, isolating each student's lab from the others. Students will be allowed to explore and experiment within the platform with their lab. The services running on the platform are not to be hidden from the students and the platform will be made as transparent as possible and free to explore. To maintain security and ensure a safe learning environment, the

platforms will adopt a least-privilege access model, where, by default, all actions will be restricted, and new accesses are introduced incrementally, with role-based access controls. This ensures that users and services will have access only to the necessary resources to complete their tasks, thereby minimizing the risk of improper setup or unauthorized access.

An emphasis is placed on safety, ensuring that no action taken in lab environments will cause any real harm to the platform, or for other concurrent laboratories. Admin users will be given wider access control to the platform, to enable visibility into the platform and, if necessary, manually control the resources running in the platform. Through these carefully considered requirements, the lab environment is designed to be secure, safe, and educational environment for students to experiment without unintended risks.

3.1.2. Lab Ordering as Self-Service

The laboratory ordering process must be integrated into the Backstage IDP. A software template [90] should be defined to streamline the ordering workflow by supporting custom actions in a pipeline-like structure. This template must automate all necessary steps involved in provisioning a laboratory environment. Additionally, the system should leverage existing user annotations and authentication mechanisms within Backstage to eliminate the need for separate user verification. Access control must be enforced by restricting provisioning capabilities to specific user groups, ensuring that only authorized students are permitted to request laboratory environments.

To make the lab provisioning process seamless and user-friendly, several critical requirements are established that prioritize ease of use and scalability. Lab environments should be easy to setup in a self-service manner, automation to maximum extent in every possible step to lower the amount of manual intervention and to ensure consistency across all provisioned lab environments.

Leveraging IaC tools with GitOps practices, will enable automated deployments, enforce configurations and maintain a repeatable process for setting up new labs. This level of automation will enable users with limited technical expertise to order laboratory environments without unnecessary friction. Laboratories should be deployed in declarative manner, and commands should be idempotent and preventive to accidentally or intentionally deploying an unlimited number of lab environments for students. Lastly, lab ordering process should not be made available to everyone, but only for a specific group of customers.

The laboratory exercises should be designed to provide a clear, step-by-step guide that supports learners in completing practical tasks within the learning platform. By following the practices used in existing learning platforms, the guide should emphasize hands-on progressive learning, where exercises initially start as introductory, and gradually increase in complexity. The provision of the guide should be integrated into the self-service process, reducing the need for additional processes, and should include contextual learning, provide the commands to run for hands-on experience, and expected outcomes to support learner understanding. To ensure usability, the guide must be easy to follow and tested within the same environment in which learners are expected to operate.

3.1.3. Chaos Engineering

The selection of a suitable chaos engineering tool for the platform should be guided by a thorough evaluation of several key factors. Firstly, the tool should demonstrate long-term sustainability, have active community engagement, be cloud native, and support command-line usage to align with developer workflows. Microsoft Azure chaos engineering offering should also be evaluated for use, as most cloud services offer levels of abstraction that can ease the use and they are tightly integrated with other cloud services.

To ensure the laboratory exercises deliver the most value, the requirements for chaos engineering experiments must be carefully defined. Rather than attempting to cover every possible failure scenario, the focus should be on a limited set of key faults. By narrowing the scope to a few critical failure modes, the labs can foster a deeper and more focused understanding of system behavior under stress. These experiments should be thoughtfully designed to run effectively against the deployed resources, reinforcing practical, hands-on exploration.

The complexity of the chaos experiments should increase gradually over time. Piloting lab should start with simpler failure scenarios, such as basic resource failure or latency faults, and in the future progressively build toward more complex faults with distributed services. This ensures that students gain foundational knowledge before engaging with more challenging chaos experiments, creating a structured learning path through the increasing complexity of the failure modes being explored.

Before conducting any chaos engineering experiments, the introduced reference application should undergo a thorough software risk assessment. This assessment is essential to proactively identify and document the potential failure modes. Understanding these failure modes in advance ensures that chaos engineering is purposeful and targeted, rather than arbitrary. By systematically exploring the weak points of the reference application, the lab can prioritize the failure scenarios that are to be tested. This process lays the foundation for meaningful chaos experiments and offers a valuable insights into chaos engineering for students.

3.1.4. Observability

When choosing the appropriate monitoring solutions for the learning platform, cloud native solutions and cloud provider offerings should be first for evaluation. The chosen monitoring and observability solutions should provide a holistic view of the learning platform. Important areas include the health and performance of the platform, scalability, and adoption. In addition to service-level metrics, the ability to monitor non-service-related indicators such as platform costs and laboratory exercise completion or turnover rates is equally important.

When determining the services and metrics to monitor during lab exercises, the focus should be on a key set of metrics that capture failures and performance degradation resulting from the chaos experiments within the labs. The lab should produce rich observability data and logs indicating the progress of fault injections, monitoring metrics like availability, latency, and error rates will enable students to assess the resilience and behavior of the reference applications during experimentation.

3.2. Solutions

After defining the requirements for the different components of the laboratory learning platform, a wide range of possible solutions will be explored. Each option was evaluated to identify the most suitable choice for the given requirements. Whenever possible, single-solution approaches were prioritized over multiple alternatives, as they inherently reduce complexity and support a more streamlined implementation.

3.2.1. Lab Engine

Azure Kubernetes Service offers a managed Kubernetes service on top of Azure cloud [103], enabling the deployment of various types of Kubernetes clusters to the cloud, primarily with either Azure-managed or customer-managed control planes. Azure managed control-plane would take care of the core Kubernetes services like Service discovery and Domain Name Service (DNS) within the cluster, and also orchestration of application workloads within the clusters. Kubernetes can be deployed also as a self-managed cluster where developers are responsible for managing the control-plane, but get full control. AKS clusters are deployed with nodepools that include multiple high-available VMs for running running Kubernetes control-plane and running workloads. Multiple nodepools can be configured they include options for choosing between different VM sizes and operating systems. AKS manages these nodes, scaling the amount of nodes up as more workloads are allocated in the cluster.

AKS introduces a steep learning curve that can be challenging to navigate, especially for newcomers. Setting up a lab engine using AKS requires a carefully designed RBAC model to manage user access. Additionally, deploying resources in AKS is non-trivial, as it involves writing Kubernetes manifest files. To reduce manual effort and minimize errors, this process should be automated as much as possible. Additionally, AKS may feel overwhelming to students who are unfamiliar with Kubernetes. Each individual lab would require multiple Kubernetes resources to be dynamically created during the provisioning phase. To reduce this complexity, the underlying Kubernetes infrastructure should be abstracted from students as much as possible, allowing them to focus on the learning objectives rather than the platform itself

Azure Container Platform (ACP) is a fully managed container hosting service that abstract the infrastructure complexity from the user. It allows users to deploy cloud native containerized workloads without needing to manage low-level deployment resources or control plane components. ACP handles infrastructure provisioning automatically and exposes the defined service endpoint with minimal configuration, offering a streamlined approach to hosting containerized applications. ACP enjoys the same benefits of high availability and scalability as Kubernetes does, while ignoring many complexities of Kubernetes.

ACP is a suitable platform for laboratory environments where the entire cluster is dedicated to a single lab. All services can be deployed as tasks into the cluster, and users access the lab through a single endpoint. By eliminating unnecessary complexity, ACP significantly lowers the barrier to entry, but it lacks controls such as RBAC to enable granular use of the shared platform. This is problematic because chaos-engineering safeguards cannot be enforced, and the blast radius of fault

injections becomes unmanageable. Additionally, some cloud-native tools designed for Kubernetes do not function properly with ACP clusters.

Azure App Service is a fully managed web application hosting platform that follows a plug-and-play model [104]. It supports deploying applications as container images and can autoscale workloads based on load and schedule.

While AKS offers greater flexibility and potential cost savings, Azure App Service provides a simpler and more user-friendly experience. It includes built-in support for features such as automatic scaling, and application diagnostics. This makes it a compelling choice for scenarios where ease of use, and minimal operational overhead are prioritized over fine-grained control. Additionally, App Services are logically separated within Azure Subscriptions using resource groups by default, and access control must be defined separately with Azure RBAC.

3.2.2. Reference Application

Self Built Container App: The laboratory environment could benefit from a purpose-built, containerized reference application tailored specifically for chaos engineering. The objective would be to develop a scalable web application with meaningful components that remains relevant for future labs and adaptable to various learning objectives. However, the focus of this exercise is not to build a perfect reference application, but rather one that is functional and sufficient for running chaos engineering experiments.

For the reference application itself, it is sufficient to support scenarios such as fetching some data from integrated storage and simulating specific faults that can be visualized in the user interface. This functionality ensures that chaos experiments are illustrative and provide actionable insights without requiring an overly complex or time-consuming development process.

Hello World Example App: The laboratory exercise could use open-source applications designed as reference examples for learning purposes. Ideally, the application should be provided as a container image to enable effortless deployment. However, smaller applications can also be used, as they are often easy to containerize and still serve as effective learning tools. A very simple hello-world application can work as a POC for the first piloting laboratory, this application should include at least two distinct services that can be injected with fault. For example, the Guestbook reference application from Kubernetes [105] includes a simple front-end service that connects to a database to maintain a list of guests, and could serve as a suitable reference application.

Bring Your Own Application Model: Container supporting platform provides flexibility to replace the reference application with any other application as needed. Ideally, this enables students to bring their own applications into the lab environment to run chaos engineering experiments against them. This approach is particularly valuable, as students are most familiar with their own solutions and are therefore better equipped to formulate meaningful hypotheses and observe relevant failure modes. This approach is more complex to implement than the previous two, as users would need to provide their own application images to the platform prior to ordering a lab. It also requires continuous effort to make new images available.

3.2.3. GitOps Tools

FluxCD is a GitOps continuous delivery tool originally developed by Weaveworks [106], and is considered the first tool developed to implement the GitOps methodology with automated deployments in Kubernetes clusters. Flux is deployed as a set of controllers within a Kubernetes cluster, using either the Flux CLI or Helm charts. Once deployed, Flux operator runs continuously inside the cluster, monitoring connected Git repositories for changes and mirrors those changes with the live cluster state. This follows a pull-based deployment model, ensuring that the desired state defined in Git is always enforced within the cluster.

In 2024, the original creator and primary corporate sponsor of Flux, Weaveworks, was shut down due to increasing business challenges [107]. Despite the loss of its founding company, Flux has continued to thrive, thanks to its graduated status under the CNCF and strong support from the broader cloud native community. Most cloud providers and new contributors have sustained active development and maintenance of the project, ensuring its ongoing stability and adoption in the future.

ArgoCD is a leading GitOps-based continuous delivery tool [108], specifically designed for pull-based deployments in Kubernetes clusters. Compared to FluxCD, ArgoCD provides several advanced features, including a built-in user interface that provides real-time visibility into application states, visual diffing between desired and actual states, and manual synchronization capabilities. It is typically deployed using Helm charts and operates as a single, bundled application within the cluster. ArgoCD is a CNCF graduated project with a large and highly active community, ArgoCD demonstrates strong adoption across industries, backed by higher community engagement and it has significantly more adopters based on having ten times more contributors in Github. The sustainability and rich integration ecosystem make ArgoCD a preferred choice for organizations embracing continuous delivery with cloud native tools.

3.2.4. Chaos Engineering Frameworks

Azure Chaos Studio is a native Azure service designed for building and executing chaos experiments against a selection of Azure resources [109]. It supports two types of faults: service-direct faults, which target Azure services directly, and agent-based faults, which require an agent installed on virtual machines to simulate in-guest failures, such as increased CPU pressure or terminating random processes within the VM. Chaos experiments in ACS are treated as standard Azure resources and would typically reside within the same resource group as the corresponding lab environment.

ACS provides an interface for managing chaos experiments through command-line operations. Users can create, modify, delete, start, stop, and monitor experiments by submitting structured input that defines the experiment type and its target resources. It is important to note that while a chaos experiment is running in ACS, usage is billed by the minute. The service supports a wide range of fault types in various Azure services, as detailed in Table 4. Furthermore, when using AKS, the Chaos Mesh tool must be installed in the cluster to enable fault injection targeting resources within the cluster.

Table 4. ACS chaos experiment offering.

Fault type	**AKS Chaos Mesh Faults**
Applicable resource	Azure Kubernetes Service (AKS)
Description	Injects faults at the Kubernetes control plane level, targeting network, pods, and system resources within AKS clusters.
Examples of Faults	DNS Chaos, Pod Chaos, Network Chaos, HTTP Chaos IO Chaos, Stress Chaos
Prerequisites	Chaos Mesh must be deployed in the AKS cluster

Fault type	**Agent-based Faults**
Applicable resource	Virtual Machines (VMs), VM Scale Sets
Description	Injects faults directly on VMs by running an agent within the VM to simulate resource strain and disruptions in CPU, memory, network, and disk.
Examples of Faults	CPU Pressure, Network Disconnect, Kill Process, Disk IO Pressure, Time Change
Prerequisites	Requires installation of Chaos Studio Agent binaries

Fault type	**App Service Faults**
Applicable resource	App Service
Description	Applies service-level disruptions specifically to Azure App Services to simulateissues such as downtime, scaling failures, and resource constraints.
Examples of Faults	Stop App Service, Restart App Service, Scale Out/ Scale In, CPU Stress, Memory Pressure
Prerequisites	None

Chaos Toolkit is a CNCF listed [51] open source chaos engineering framework written in Python where chaos experiments are declaratively defined as code [110]. It offers a flexible and powerful set of extensions to implement, manage and automate chaos experiments. Chaos Toolkit is run with command-line interface tool which makes it efficient tool for engineers to run chaos tests directly from their terminal.

The Chaos Toolkit offers seamless integration with Azure through a dedicated extension, enhancing its versatility for teams leveraging Azure's cloud ecosystem. This extension supports a range of Azure-specific faults, from service targeted disruptions with Azure Native Services to dedicated Kubernetes faults in AKS. Additionally, observability extensions enable data import and export across various monitoring platforms during chaos experiments, helping teams maintain visibility throughout testing. Further plugin interfaces allow the Chaos Toolkit to publish detailed reports on experiment outcomes, which can be forwarded to various notification channels, ensuring teams stay informed and can act quickly on insights gained.

While the last release of Chaos Toolkit occurred over a year ago, At the time of writing (25.03.2025), the Chaos Toolkit repository had 18 active contributors in Github, this is a moderate amount of contributors and shows some amount of

community involvement for the tool. This lack of ongoing development makes it a less attractive option for adoption in modern chaos engineering workflows.

Chaos Mesh is an open source, cloud native chaos engineering tool for running chaos experiments in Kubernetes clusters that is also owned by the CNCF. It is deployed with Custom Resource Definitions (CRD) to a Kubernetes cluster and can be deployed using public Helm charts. Chaos Mesh comes with Chaos Daemon that has privileged access to manipulate pods in the cluster, a chaos controller is responsible for scheduling and includes multiple controllers for controlling the chaos experiments within the cluster, and Chaos Dashboard for observing and manipulating chaos experiments.

It supports a wide range of fault simulations, such as DNS faults, network faults, time faults, and stress faults. To use Chaos Mesh effectively, proper Kubernetes RBAC should be implemented to manage chaos experiments. This ensures that the Chaos Dashboard only displays faults that target the student's own laboratory environment, strictly prohibiting unauthorized faults against other resources. Chaos experiments are defined as several different chaos CRDs and can be scheduled from the command line or from built-in dashboard.

3.2.5. Chaos Engineering Exercise

In order to effectively impart the foundational principles of reliability engineering, the chaos engineering laboratory is designed to focus on a single, well-defined chaos journey per exercise. This approach ensures that students can fully immerse themselves in each scenario and internalize the key concepts associated with resilience and fault tolerance. The laboratory framework is structured around multiple chaos journeys, each offering a unique perspective on system reliability. Specifically, three distinct journeys have been identified:

- **Journey 1**: Simulate a service degradation scenario by introducing controlled latency in network communication between microservices.
- **Journey 2**: Combine a broken service with incorrect network configuration, thereby requiring students to detect and address two simultaneous issues.
- **Journey 3**: Inject faults preemptively without disclosing the root cause, challenging students to identify and resolve the issue through systematic investigation.

3.2.6. Observability Tools

The observability tools selected for the laboratory platform should be based on the comparative analysis presented in Table 3. These tools offer a diverse set of capabilities for collecting metrics, logs, and traces, as well as for visualizing the performance of the laboratory engine. To gain comprehensive visibility into the learning platform and laboratory environments, multiple tools can be combined to form a holistic observability stack.

3.2.7. Lab Guide

A stateful lab guide is one where the platform tracks the progression of each user and presents it in a comprehensive way. Implementing this requires a database to persist user-specific data and associate it with individual user identities. The application must be aware of user actions in order to determine progress throughout the exercise. Additionally, to offer a seamless experience similar to commercial platforms [89], the system may need to support interactive shell environments that align with step-by-step instructions.

In contrast, a stateless lab guide can be implemented in simpler ways. One approach is to provide a shared documentation file used by all users, which contains generic, non-personalized instructions for completing the lab. User-specific information, such as the deployed namespace or security tokens be delivered separately, for example, as output from the software template. In this model, each student follows the same guide, optionally forking or annotating their own version as they progress.

A more dynamic approach involves generating a personalized lab guide during the self-service lab ordering process. This requires implementing a custom action in the software template to automate the creation and population of the guide with user-specific details. With this method, each student receives a tailored guide, and the system can provide a direct link to it as part of the template output. This also enables tracking of individual student progress through the generated guides, offering better visibility into lab usage and engagement.

3.2.8. Role-Based Access Control

There are multiple ways to give access for the user resources in lab environments. To ensure consistent and secure access control, RBAC configurations should be integrated into the IaC tools used for deploying lab environments, applying permissions automatically during setup. Alternatively, a shared RBAC template can be maintained within the subscription and referenced when provisioning new labs, ensuring uniform policies across all environments. It is crucial to balance security and cost control with the flexibility students need to explore and complete tasks creatively. Overly strict restrictions may simplify management but risk stifling engagement and problem-solving. Additionally, allowing students to decommission their lab environments after use promotes autonomy and helps eliminate unnecessary costs, supporting efficient resource utilization across the RE Lab infrastructure.

All Access Model would grant students full access to all laboratory resources within the lab engine, including permissions to modify or delete resources. Such unrestricted access could lead to unintended consequences, such as students prematurely ending their exercises by causing the lab environment to enter an undesirable, irrecoverable state. Furthermore, if multiple users share the same subscription, this lack of access control could result in students accidentally or intentionally disrupting resources that do not belong to them.

In scenarios where labs are deployed in decentralized subscriptions, like in the Bring Your Own Subscription model, students probably would require full access to the laboratory resources, to ensure they can complete the lab exercises and eventually

destroy the resources. Controlling the access of students in these scenarios is impractical and should not be considered in any way.

Azure RBAC is a managed security mechanism used to define access to Azure resources by assigning roles to users, groups, or service principals. It allows for fine-grained control over who can perform specific actions within a subscription. In the context of the learning platform, Azure RBAC can be used to restrict student access to essential resources. For instance, access to AKS resources can be limited to read-only, preventing students from modifying the configuration of the lab engine. Similarly, creation or deletion of other cloud resources within the shared subscription should be prohibited. Azure provides a set of predefined roles for common access scenarios, and also supports the creation of custom roles to tailor access permissions according to specific requirements.

Kubernetes RBAC Operating a Kubernetes-based lab engine requires managing access within the cluster, as it would host all the lab resources. This necessitates the deployment of access controls at the cluster control level, either concurrently with or prior to the deployment of lab instances. These controls would ensure that students can only view and interact with their own lab resources. Additionally, Kubernetes control tools would be configured to display only the shared and owned resources associated with a student in the cluster.

The cluster would be logically separated into different namespaces, with each student assigned their own namespace to deploy their lab resources. Student built chaos experiments could only target resources within a student's namespace, and other actions would be restricted based on the specific security policies of the lab exercise. Students could also have view-only access to the default namespace, where shared resources are deployed, but they would be unable to modify or interact with these resources. Administrators, on the other hand, would have broader permissions to manage, scale, or destroy any resources within the laboratory cluster.

3.3. Decisions

This section outlines the key architectural and technological decisions that will be implemented during this thesis. Each choice is based on identified requirements, a careful evaluation of alternatives, and their alignment with the goals of the learning platform. By considering the strengths, trade-offs, and integration potential of each option, the aim is to establish a reliable and maintainable foundation for implementation.

3.3.1. Lab Engine

A flexible, highly available, and cost-effective solution that could be deployed in Azure was required for the lab engine. It needed to support chaos experiments and be user-friendly. After careful consideration, AKS [103] was chosen as the lab engine for the learning platform. Althought it met all but one of our requirements, it demands higher technological skill level compared to its competitor, Azure App Service.

Firstly, Kubernetes is a cloud native tool that is backed with huge support from its community and maintainers, it is aligned with all set requirements for running the learning platform. Also, since many other cloud-native tools are designed to operate within Kubernetes, a wide range of different cloud-native technologies can be leveraged within the learning platform through this choice.

Secondly, AKS is a more flexible tool. Kubernetes provides unparalleled flexibility in designing the lab infrastructure, offering a vast array of resources that can be tailored to fit any exercises needs, including defining custom Kubernetes RBAC. Moreover, the ability to use Helm as a packaging manager simplifies the processes of laboratory exercise provisioning and management of other resources within the Kubernetes cluster.

Thirdly, operational expenses can be reduced and greater control can be achieved through the use of AKS. The control plane is created at no upfront cost, is fully managed, and does not require its configuration to be handled. The computing resources for any node in the cluster can be controlled, and the number of nodes can be auto-scaled by AKS based on load. Additionally, the cluster can be configured to scale down computational resources during idle periods or even shut down entirely during off-hours when student activity is not expected.

Taking into account these factors, along with the high availability inherent to Kubernetes clusters, AKS emerged as the superior choice for the lab engine. Despite this decision, The flexibility to deploy multiple different lab engines side by side for diverse use cases in the future is retained. However, for this study, a single engine suffices. The cluster is designed to operate with a capacity ranging from zero to five user nodes. Zero when the laboratory engine is in hibernation (e.g., during out-of-office hours), and up to five at full throttle during peak traffic, such as when multiple students are concurrently running their laboratories in the engine. The system node pool must be configured at minimum of two active nodes at all times and it will be given one more node for auto-scaling if needed. VM sizes between the node pools differ as Kubernetes control plane services with smaller CPUs, but operators like ArgoCD, that are deployed to the user nodes demand much more CPU quota. Details of the master node specifications are provided in Appendix 1 Table 6.

3.3.2. Reference Application

Due to the nature of Kubernetes, it was decided to develop and deploy an laboratory reference application as a containerized solution consisting of several services, including a database. It was concluded that a reference application tailored to the specific use case of the chaos lab should be built, thereby avoiding the unnecessary complexity of larger applications. While many potential reference applications were reviewed, they were ultimately too complex for the chaos engineering use case, where the goal is to demonstrate chaos engineering principles against a simple microservice. Adapting third-party reference applications to meet our requirements would necessitate maintaining forks of these solutions, potentially resulting in significant overhead. Considering these factors, it was chosen to start with a simple application that encompasses multiple microservices, can be deployed to an AKS cluster, and is easy to maintain while also scalable for use in future laboratories.

3.3.3. GitOps Tools

For automated deployments, ArgoCD has been selected as the Kubernetes operator to synchronize the desired state of the laboratories with the lab engine cluster. It was chosen due to its adherence to GitOps principles and the support of a growing, sustainable community. By utilizing GitOps, repository history can be managed, configuration changes can be reverted, and the learning platform can be scaled by publishing new laboratory exercises as code, all while leveraging familiar developer tools like Git.

Laboratory environment deployments are enabled through ArgoCD by using Helm charts, which are parameterized with values specific to each student's configuration. A significant benefit is provided, as the complexity of deploying laboratory environments within a Kubernetes cluster is abstracted away from the students. Through these capabilities, a sustainable and automated deployment tool is provided to support the laboratory ordering model as a self-service platform.

3.3.4. Lab Guide

For the lab guide, a stateless design is chosen that generates guides dynamically during the lab ordering process. This approach simplifies the overall learning platform by eliminating the need for databases or additional state tracking processes. Instead, users are responsible for tracking their own progress within the guide as they work through the laboratory exercise. Jira [111] tickets are used to provide laboratory guides.

The decision to deliver the lab guides as Jira tickets adds several benefits. First, it allows the guides to be easily cloned, providing a seamless way to replicate exercises for multiple users. Second, using ticket subtasks for each step in the laboratory exercise offers a clear, step-by-step structure that users can follow at their own pace. Furthermore, a lab guide ticket can be assigned to a user, and the platform usage can be tracked using various ticket metrics such as lab duration and adoption rates. This setup makes it straightforward to view who has completed which exercises and to assess overall progress.

3.3.5. Chaos Engineering

The Chaos Mesh service will be installed within the Kubernetes cluster. This decision was straightforward once the lab engine was chosen. Chaos Mesh had already been evaluated for its wide range of chaos experiments, which can be combined to create comprehensive fault scenarios against applications in the lab environment. Chaos Mesh is a cloud native chaos engineering tool that is designed to be used with Kubernetes.

The installation of Chaos Mesh also enables the use of Azure Chaos Studio as a chaos engineering platform, as one of the requirements to use ACS with AKS is to have Chaos Mesh installed. However, fully instrumenting ACS would require deployments using conventional IaC tools, and the operational costs associated with scheduling experiments in ACS are significant. Therefore, ACS will be excluded from the scope

of this study, and only the use of plain Chaos Mesh will be considered from this point forward.

For the piloting chaos laboratory exercise, the Journey 1 chaos was chosen to built the basis for the laboratory exercise. In this chaos journey, the reference application begins in a healthy state, and users are tasked with injecting controlled latency faults that can be observed from monitoring tools. The reference application should incorporate a microservice architecture designed for fault injection experiments, and include an interactive user interface that visually demonstrates the faults in real time.

3.3.6. Observability Tools

To achieve performance monitoring from the workloads running in the laboratory engine, OpenTelemetry [71] SDK will be used to instrument the reference applications. This choice is driven by the fact that OpenTelemetry is a highly adopted observability tool in the CNCF community. It also streamlines the workflow to collect metrics, traces and logs in a unified way. Application-level instrumentation will be implemented using the OpenTelemetry SDK for Node.js [3], exporters will be defined in application code and the container responsible for running the application will collect and export the telemetry data to other monitoring tools.

Prometheus and Jaeger are chosen as the observability tools to visualize metrics and traces inside the learning platform. Prometheus can scrape all laboratory environments for user specific metrics and also measure the health of the Kubernetes cluster and can be used to visualize the healths from all these levels, also during the fault injection. Jaeger can gather distributed traces from services and the distributed traces can be used to visualize badly acting services in distributed service fault injection. Both of these services will be deployed as containers to the lab engine.

Since the lab engine is deployed on AKS, its built-in cost monitoring features will be used to approximate platform expenses and track actual usage costs during the evaluation phase.

3.3.7. Role-Based Access Control

Azure RBAC is implemented to ensure that students have granular access to their lab resources while only being granted view access to shared resources. This means that while students can modify the lab resources assigned to them, they are not able to control any services that are shared across the environment.

Access control is achieved by assigning each student least-privilege roles that are limited to their individual resource groups, ensuring that they can interact only with their designated Azure resources in a controlled manner. In addition, Kubernetes RBAC is employed to restrict students' access via the Kubernetes API to their specific namespaces within the cluster. This prevents any unauthorized actions in the default namespace or in namespaces assigned to other students.

[3] https://opentelemetry.io/docs/languages/js/getting-started/nodejs/

Administrators, on the other hand, are given full access to all resource groups in Azure and across all namespaces in Kubernetes. This comprehensive access allows for effective oversight and management of the entire environment, ensuring that the system remains secure and properly maintained while adhering to the principle of least privilege.

4. IMPLEMENTATION

After a thorough analysis of the technologies and methodologies selected for the learning platform, the focus of the study has been refined to emphasize the key elements required to deliver the initial release of the pilot laboratory environment. In this chapter, the implementation of the learning platform is initiated, beginning with the design of a solution architecture that defines all processes involved in the learning platform in sufficient detail. Following the architectural design, the practical implementation of the solution is carried out, with the chosen approaches explained at each stage of development. At the end of this section, the completed platform is evaluated based on several measured metrics that provide insight into the research questions outlined at the beginning of this study.

4.1. Development of Lab Engine

When transitioning to the implementation phase of my study, the focus shifts to translating theoretical concepts into, practical working systems. This phase consists of several key steps that ensure a structured and iterative approach to achieving the study's goals.

Minikube[4] was used locally to test the first implementations of the reference application and the Chaos Mesh framework. Minikube is a lightweight, Kubernetes environment running on local machine, that deploys Kubernetes core functionalities and development of infrastructure locally. The reference application is developed locally to confirm its usability and effectiveness in demonstrating reliability engineering concepts. Simultaneously, the Chaos Mesh framework is integrated and tested to verify that fault injection experiments can be executed as planned. This local testing phase ensures that any issues are identified and resolved early, minimizing disruptions during subsequent deployments. Additionally ArgoCD was deployed to minikube to verify automated laboratory deployments with Helm charts.

Once the implementation is validated locally, the next step is transition to the Azure subscription. Here, the lab engine infrastructure is declared using Terraform HCL and deployed to the target subscription in a consistent and repeatable manner. ArgoCD will be deployed first to the cluster, all other infrastructure will then be declared and ArgoCD will handle deployments automatically, this will include monitoring with Prometheus and all active lab environments. To set the stage for broader use, necessary RBAC is deployed both to Azure and Kubernetes cluster.

The lab order template will be made available once the lab engine is up and running. ArgoCD manages the delivery of laboratory environments using Helm charts. A corresponding software template will be published in Backstage IDP, enabling users to provision lab environments through a self-service interface. The IDP will handle user authentication and authorization, ensuring that only eligible users have access to the ordering template.

[4]https://minikube.sigs.k8s.io/docs/

4.2. Lab Engine Architecture

The lab engine provides the computing power required to run multiple laboratory environments simultaneously on a centralized platform. At its core, the lab engine is hosting a Kubernetes cluster, enabling the deployment of laboratory environments as a group of pods. The Kubernetes cluster is deployed on three identical virtual machine nodes, each with equal computing power. Two of these nodes are designated for Kubernetes system operations, while the third node primarily hosts the students' lab environments. Additionally, various necessary tools have been installed on the cluster, which will be analyzed in detail in the following subsections. Thanks to the flexibility of containerization, new tools can be easily added to this catalog in the future if needed. For a visual representation of the lab engine, a system architecture diagram is provided in Figure 2.

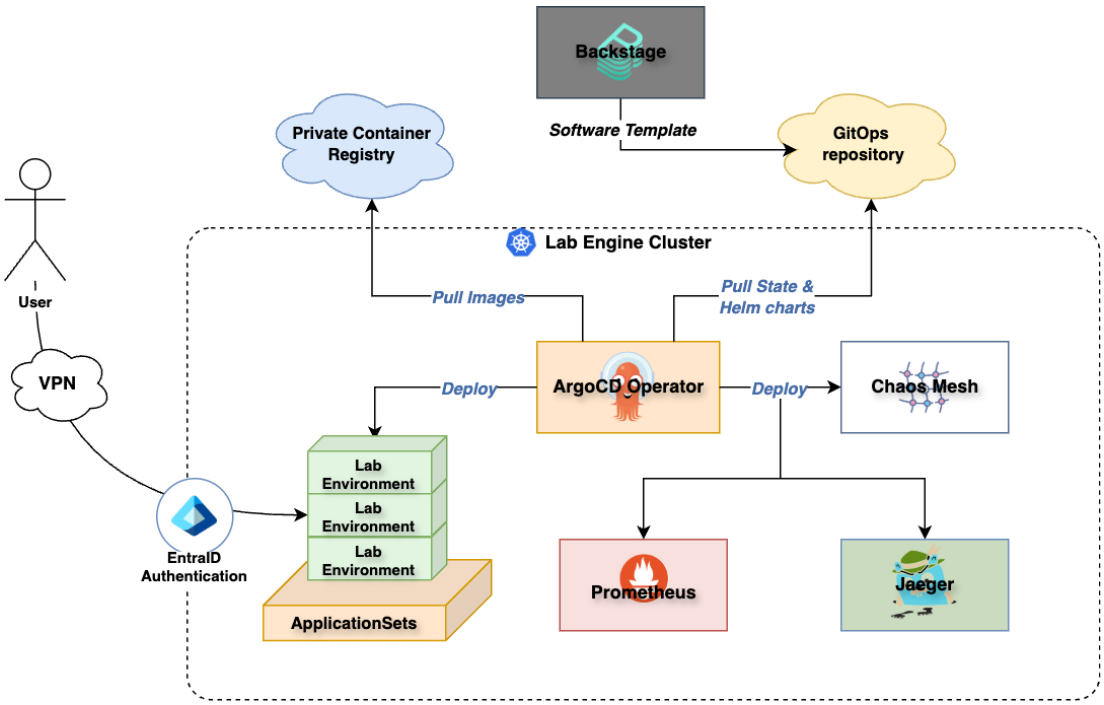


Figure 2. Overview of the lab engine architecture, including ArgoCD automated deployment model and connectivity outside the cluster for users.

4.2.1. Scalability

To run the laboratory environments in AKS, a set of virtual machine hosts need to be defined as nodes to host the Kubernetes platform, Microsoft requires that the system node pool have at least two hosts to ensure high availability of the control plane [112]. Our control plane does not need more nodes and configure the minimum and maximum of system nodes to two. The user node pool will host all laboratory environments and other lab-enabling tools. The user node pool is defined to have a minimum of zero

and a maximum of five nodes. This configuration allows the user node pool to scale horizontally based on demand in the lab engine.

To save on computing costs during out-of-office hours, nodes should be scaled to minimal necessary amount, this is possible through deletion of all resources on user node pools but can not be used with system node pools as control plane resources can not be removed without the cluster breaking. Managed AKS clusters include an additional feature that allows stopping and starting a Kubernetes cluster[5]. AKS stores all Kubernetes objects in etcd, a key-value store hosted in Kubernetes, from which they are recovered when the cluster is restarted. Stopping and restarting also applies to the system node pool, effectively scaling every Kubernetes resource down. This is particularly beneficial with some Kubernetes resources, like long-lived security tokens that would be needed to recreate after each redeploy.

4.2.2. Security

Microsoft Entra ID authentication will be enabled for AKS, to utilize the Azure's identity management framework to identify users in the lab engine. For Kubernetes cluster access, Kubelogin[6] will be used to authorize users. Kubelogin leverages Azure authentication to retrieve the necessary kubeconfig credentials, allowing users to access the Kubernetes cluster interactively with tools such as kubectl.

To manage access control within AKS, Kubernetes RBAC resources are defined with other resources in the laboratory environment during the ordering process. The RBAC manifests are defined as Helm charts within the laboratory configuration files, and ArgoCD is responsible for deploying the RBAC configurations to the lab engine.

Kubernetes RBAC for the laboratory environments is defined using two primary resource types: Roles and RoleBindings. A Role specifies a set of permissions that define which actions a user can perform within a specific API group and namespace. A RoleBinding is used to associate a Role with one or more users, thereby granting them the defined permissions within the target namespace. In cases where broader access across the entire cluster is required, a ClusterRole can be defined and bound using a ClusterRoleBinding, enabling permissions to span all namespaces.

A set of RBAC will be given for the student to enable progressing through the lab exercise. A combination of roles and cluster roles is necessary for the students to explore, but also to prevent them from accessing other laboratory environment they are not authorized to use. Specifically users will be able to: list all namespaces in the cluster, view all resources excluding secrets in the default, argo-cd and chaos-mesh namespaces, list all resources in their own namespace, access Chaos Mesh, and define and schedule chaos experiments in their namespace. A collection of Helm charts to configure Kubernetes RBAC for secure use in shared learning platform is given in Appendix 3 Figures 16–18.

[5]https://learn.microsoft.com/en-us/azure/aks/start-stop-cluster

[6]https://github.com/Azure/kubelogin

4.2.3. ArgoCD

The ArgoCD operator will be deployed in the lab engine cluster within its dedicated namespace. It will manage the automated deployment of all ordered laboratory environments in a GitOps manner, including the monitoring and chaos engineering tools. By following the pull-based GitOps methodology, ArgoCD will mirror the desired state in the Git repository to the actual Kubernetes cluster. ArgoCD operator must be configured with the necessary credentials to access the private version control. ArgoCD will be set to continuously monitor the main branch of a GitOps repository containing the configuration files for active laboratories. The directory structure of the GitOps repository and the ArgoCD mirroring logic is depicted in Figure 3. The repository will include Helm charts for all resources within an laboratory environment, configuration files for each currently active laboratory, and the ArgoCD ApplicationSet Custom Resource depicted in Appendix 5 Figure 22 defined for each available laboratory exercise. Additionally, a cluster-wide RBAC is given to each student to have basic access rights to operate in the Kubernetes cluster and is depicted in Appendix 5 Figure 23, this is done to enhance the user experience working with the lab exercises.

Each laboratory environment is deployed using parameterized Helm charts located in the helm-chart folder, which is populated with values from the configuration files located in the active-labs folder. ArgoCD can automate the deployments of individual laboratory environments using Helm charts.

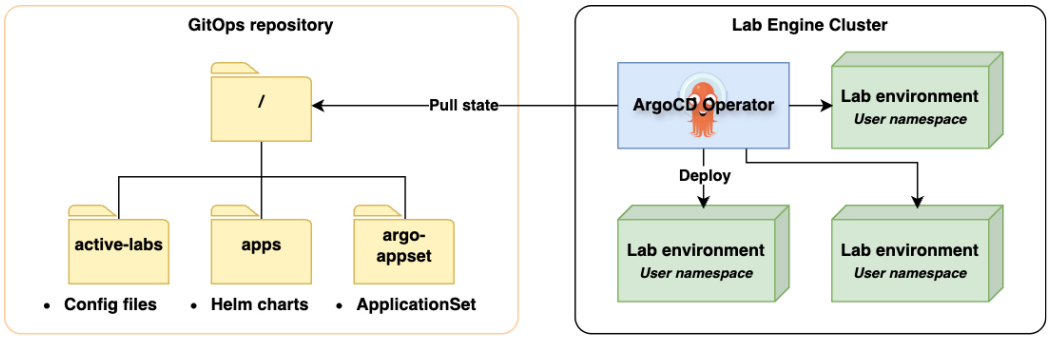


Figure 3. GitOps repository structure includes the ApplicationSet manifest, active lab config files and application Helm charts, and the ArgoCD Operator processes it to automatically deploy lab environments across cluster nodes.

Active laboratories are defined as individual JavaScript Object Notation (JSON) files, each containing all the necessary information required to create a lab instance. These JSON configurations serve as a blueprint for provisioning labs dynamically and ensuring that each laboratory session is uniquely tailored to the assigned user. The lab engine will support only one active laboratory exercise per user at a time, and the active-lab folder should be checked before creating a new lab for a student.

To personalize the laboratory experience, each configuration must include user-specific details. At a minimum, this should include:

- The student's name

- Their personal Microsoft Entra ID email, for authentication and authorization within the lab engine

Additionally, the configuration must specify which laboratory exercise the student intends to execute, along with any variant information related to the selected exercise. This flexibility ensures that different lab setups can be accommodated within the same framework while maintaining centralized control over deployments.

An ApplicationSet is a custom resource that allows defining and managing multiple ArgoCD applications dynamically based on templated configurations. Instead of manually creating individual ArgoCD applications for each deployment, an ApplicationSet can generate and manage multiple applications from a single definition, using parameters such as Git repository contents, cluster lists, or other sources. Since different laboratory exercises might require different parameters and configurations, different ApplicationSet resources are created. This approach enables multiple distinct laboratory deployment definitions to coexist within a single GitOps repository. With this setup, a single ArgoCD operator can handle deployments efficiently, applying different configurations as needed while maintaining a consistent GitOps workflow.

Laboratory exercises are defined as Helm charts, which serve as templates for deploying the necessary Kubernetes resources required for each exercise. A Helm chart is a IaC template for Kubernetes that simplifies the deployment, versioning, and management of applications by organizing configuration files and resource definitions into a structured format. It allows applications to be deployed repeatedly to different namespaces within the lab engine, enabling the same laboratory exercise to be deployed multiple times for different students with unique configurations.

4.2.4. Laboratory Lifecycle

All currently active laboratories are defined within the `/active-labs` folder of the GitOps repository. The laboratory lifecycle is managed using basic Git operations to modify the configuration files in this directory. Provisioning and teardown of labs are controlled through Git state changes and are automatically synchronized by ArgoCD. The lifecycle of a laboratory, from provisioning to teardown, consists of two simple steps:

1. A merge commit is pushed to the main branch for lab provision.
2. The commit is reverted for lab teardown.

A lab order commit will contain one configuration file, which equals to one active laboratory environment in the lab engine when merged with the GitOps repository. The configuration file is populated with metadata and parameters collected through the software template in Backstage.

ArgoCD is monitoring for changes in the GitOps repository and synchronizes the desired state of laboratories with the lab engine by deploying all necessary containers according to the ApplicationSets inside the Kubernetes cluster.

The GitOps repository maintains the ordered history of all changes made to the repository. By reverting the lab order commit, the configuration file is effectively

removed from the repository index. ArgoCD will synchronize the desired state with the Kubernetes cluster, effectively destroying all resources associated with that lab environment.

4.2.5. Container Registry

All referenced container images and Helm charts are stored in a private container registry that the lab engine cluster is able to pull images from. Managed private Azure Container Registry (ACR) will be used for the implementation with AKS cluster as these services have seamless integration, but any other container registry could be used. The Kubernetes kubelets will be assigned sufficient roles to to pull any images that exist in private registry. Containerd runtime will cache pulled images in local image repository in each node, eliminating the need to repeatedly pull the same images from ACR unless the images are updated or when cache is cleared during cluster restart. This approach optimizes performance and enables faster container startup. Kubernetes pods can be set to always pull images from ACR in the deployment config.

By hosting a private container registry, users can push their own images into the ACR and use them effectively within the laboratory environment. This expands the range of possible laboratory setups in the future, allowing users to safely test and experiment chaos engineering with their own applications while not having to worry about deploying the infrastructure or tooling.

4.2.6. Chaos Mesh

Chaos Mesh will be deployed under its own namespace, using a simple Helm chart provided with the getting started with Chaos Mesh documentation. Chaos-mesh will be defined as part of the whole infrastructure and is deployed together with initial infra. For security and lab-engine reliability, students will have scoped access to configure chaos experiments only in specified namespaces. To enhance the security and control of chaos experiment configurations, faults will only be injected into workloads within namespaces that contain the annotation `chaos-mesh.org/inject=enabled`. If this annotation is not present, Chaos Mesh controller will not try to injects faults in that namespace. This approach allows administrators to control which namespaces are affected, ensuring that critical workloads remain protected from unintended disruptions.

User permissions will be managed using an additional Chaos RBAC configuration, which will grant students permissions to conduct chaos experiments within their assigned namespaces. When a new laboratory environment is created for a student in a new namespace, corresponding Role, RoleBinding and ServiceAccount resources will be created to grant access to chaos experiments within that namespace. The ServiceAccount provides the necessary identity for Chaos Mesh, while strict access controls define which chaos experiments a student is allowed to perform and which resources they can target.

Chaos Mesh requires a secret token for authenticating each user to schedule chaos experiments either with cluster-wide or namespace permissions. Students will be given

namespace-specific controls on their ServiceAccounts, allowing them to access the dashboard and configure chaos experiments within their own laboratory environments. The tokens are created on behalf of the students during the laboratory provisioning process and are accessible to ensure the seamless use of chaos engineering tools. To maintain security, students are restricted from creating their own tokens, avoiding the risk of overly powerful access.

Chaos Mesh includes three key resources: the Dashboard, the Controller Manager, and the Daemon. The logical orchestration of these components is available in Figure 4.

Chaos Dashboard provides an intuitive graphical interface for configuring chaos experiments on namespace pods. Besides the dashboards, users can also configure chaos experiments as Kubernetes manifests and apply them using command-line tools such as kubectl. Controller Manager is responsible for orchestrating chaos experiments by managing the lifecycle of chaos resources, reconciling experiment specifications, and ensuring that the desired state of chaos is applied to the targeted workloads. It continuously monitors the running experiments and ensures that they align with the defined policies and parameters. Chaos Daemon runs on each node within the cluster and is responsible for injecting faults directly into the pods at the system or network level. It executes the actual chaos experiments, such as CPU stress tests, network latency injections, and disk I/O disruptions, based on the specifications provided by the Controller Manager.

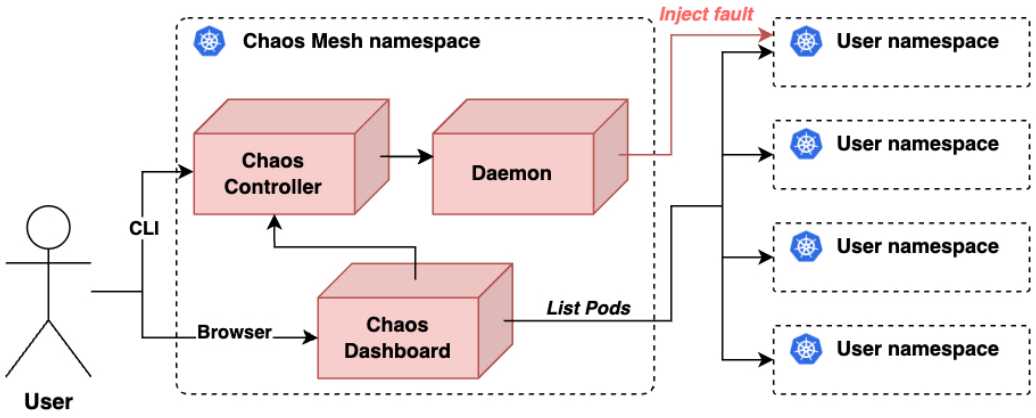


Figure 4. Flowchart for configuring chaos experiments using the Chaos Mesh dashboard, including all necessary components in the process.

4.2.7. Monitoring

The monitoring stack includes both Jaeger [73] and Prometheus [36] to provide holistic observability over the laboratory engine. Both services are deployed as containers in their own namespaces within the lab engine cluster. Additionally, Kubernetes Services are deployed to expose the pods and provide access to the user interfaces of both

tools. A custom Kube-state-metrics[7] service is also deployed to collect metrics from the cluster itself.

Prometheus is configured to scrape metrics from all laboratory environment pods, the Kube-state-metrics service, and from itself. The Prometheus configuration is provided in Appendix 4 Figure 20.

Jaeger is a push-based monitoring tool and requires minimal configuration. It only needs a designated namespace and service name, both of which are set to "jaeger" in the Kubernetes manifests. These values are later used by the reference applications, as KubeDNS resolves the Jaeger service's IP address based on its namespace and service name. The DNS name and other trace configuration used in the OpenTelemetry instrumentation of the reference application are shown in Appendix 4 Figure 21.

This monitoring stack is used to assess the health of the learning platform and individual laboratory environments. It will also serve as a foundation for evaluating the overall implementation of the learning platform. The observability architecture and telemetry flow within the lab engine are illustrated in Figure 5.

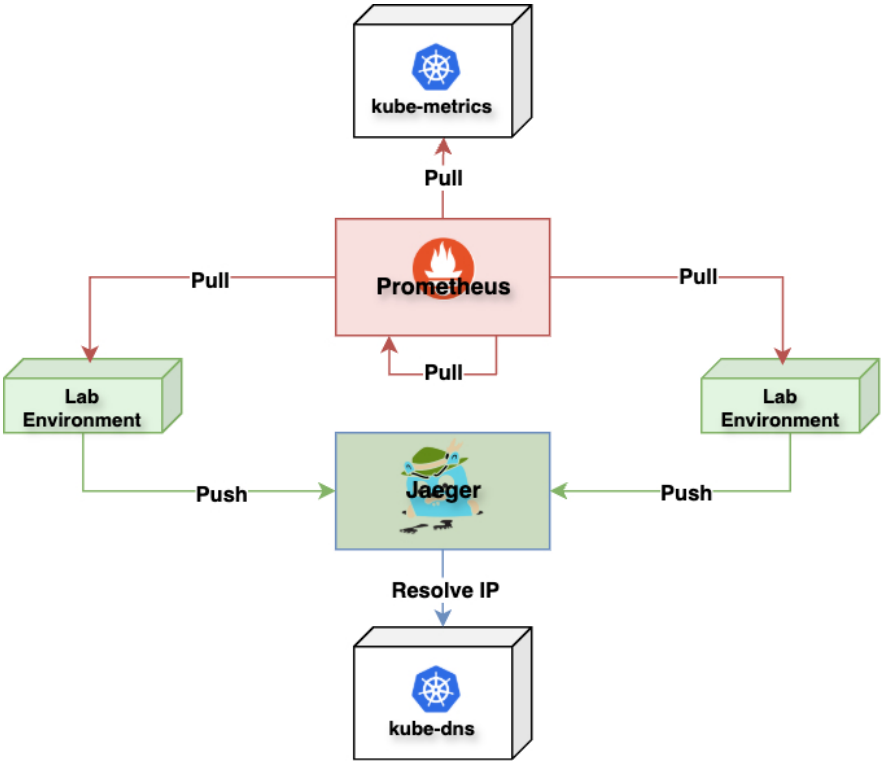


Figure 5. This monitoring stack diagram illustrates how Prometheus scrapes metrics from all configured services in a pull-based model. On the other hand, Jaeger uses a push-based model and relies on KubeDNS for service discovery of laboratory pods.

[7] https://github.com/kubernetes/kube-state-metrics

4.3. Reference Application

A reference application will be deployed with the chaos engineering laboratory. A concrete application is essential for students to build meaningful chaos engineering experiments. By developing an reference solution, the entire architecture and user interface can be designed to meet the specific requirements of the pilot lab. Delivering my own reference application also ensures scalability for subsequent laboratory exercises that could benefit from a similar solution. Additionally, by building an reference application for the laboratory, the dependencies and limitations of third-party applications are avoided, and the setup is kept as simple as possible while ensuring suitability for various chaos experiments.

4.3.1. Micro-Service Architecture

The laboratory environment consists of three different microservices built on top of containers. Having multiple micro-services gives the opporunity to decouple the application logic into three distinct layers, with each layer being responsible for a small portion of the overall application. The microservice architecture also allows students to focus on disruptions and faults in individual parts of the application, which are both measurable and visible in the user interface. The infrastructure is implemented using a deployment with one replica for each application and a service for each deployment to enable communication between the services in the Kubernetes cluster. More specifically, a service of type LoadBalancer for the frontend will be implemented to expose the containers to users outside the cluster, ensuring seamless access to the user interface. It should be mentioned that exposing services with only type LoadBalancers is a feature of managed AKS and might not exist in other solutions. The resource quotas for different services in the laboratory environment are configured according to the values shown in Table 5. These quota are applied to prevent laboratory environments from consuming excessive resources during fault injection experiments.

Service	vCPU request	vCPU limit	RAM request	RAM limit
Frontend	0,2	0,4	512Mi	512Mi
Backend	0,1	0,2	128Mi	128Mi
Redis	0,1	0,2	128Mi	128Mi

Table 5. Resource quotas and hard limits configured for all services in the lab environment to prevent overconsumption of resources in the lab engine.

The microservice architecture of the reference application is depicted in Figure 6. It includes a frontend service that serves the user interface and has reverse proxy capabilities to route API requests to backend service. The backend is a simple service for routing request between frontend and database, which runs default Redis in-memory database image[8]. The reference application includes a simple CRUD app that retrieves response times from the Redis service, stores response times from new requests in the database, and displays them through graphical interface for the user.

[8]https://hub.docker.com/redis

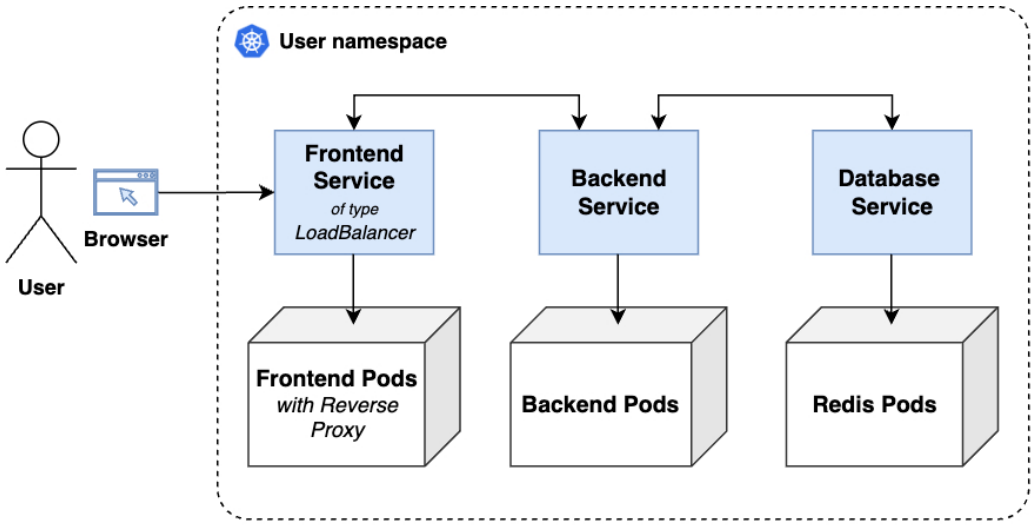


Figure 6. The microservice architecture of the reference application.

4.3.2. FMEA for Reference Application

Understanding the different ways in which the reference application might fail during operation is essential prior to initiating chaos engineering experiments. FMEA [74] will be done for the reference app to systematically identify potential failure modes and prioritize them based on risk to understand which failure modes to enable during the chaos engineering exercise.

Prior to the analysis, cluster-wide faults will be excluded, and it will be assumed that the cluster remains healthy throughout the experiment. While this does not reflect the reality of production environments, this simplification is made to narrow the scope of the initial chaos engineering exercise. As a result, the focus will be limited to faults affecting the microservice architecture of the reference application contained within a single laboratory environment.

For each service in the reference application, a simple FMEA will be conducted that identifies possible failure modes, describes their effects on the user, and ranks them by severity on a scale of 1 to 10. The results are presented in Appendix 2. These failure modes are then later used when planning chaos engineering experiments in the pilot lab exercise.

4.3.3. User Interface

The reference app user interface will be made available for the users from web browser. Azure Load Balancer service will provide the frontend services with an external IP address that are accessible outside the Kubernetes cluster. The frontend will include an interactive graph that draws Redis database latency as a function of attempts. This graph illustrates deviations in response times observed during chaos engineering experiments, making it easier to monitor and understand the effects as they occur.

Additionally, the display will show the latest database response time, with its color indicating the overall health of the reference application. The user interface presenting a set of healthy response times from Redis database is depicted in Figure 7.

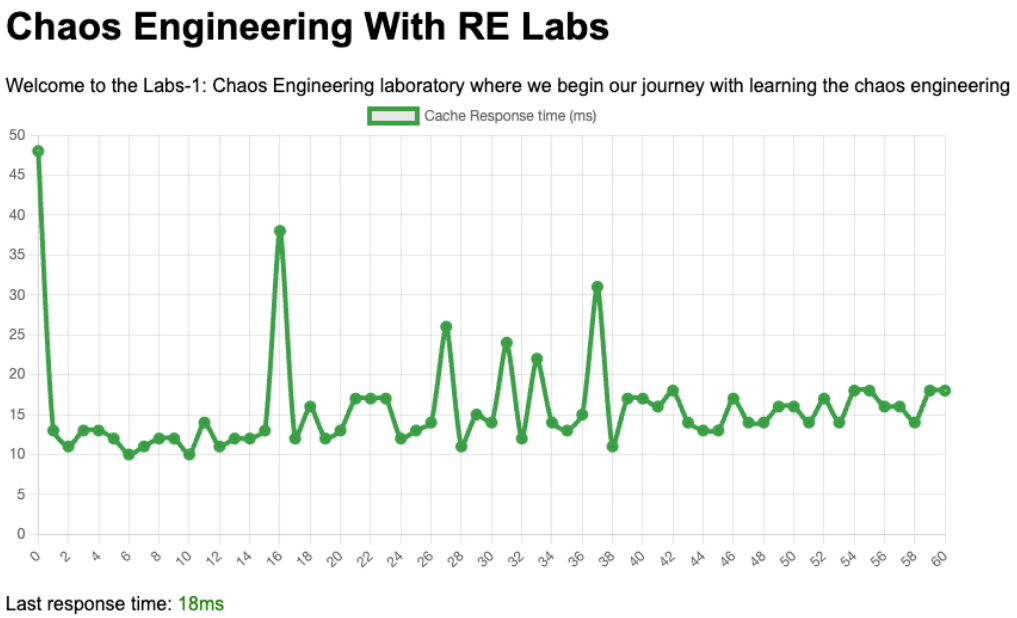


Figure 7. Reference App User Interface showing 60 healthy requests to the Redis database service.

4.4. Self-Service Template

During the methodology phase, it was decided that the ArgoCD operator would be used to deploy laboratory environments through a GitOps repository. To modify the state of the GitOps repository, a specific set of templated actions must be built and logically arranged in the correct order to form an ordering pipeline. This pipeline will be built with Backstage Software Templates.

The access to the ordering template is being restricted to a specific group of people. This group is represented as a distinct Group entity in Backstage and includes all student entities. In addition, the number of active laboratories will be limited to one per user. All actions in the self-service template for lab ordering needed to be custom built, except the input and output stages, and include the use of various tooling, from text manipulation to git implementation. The complete self-service template is built from the following actions.

Input: At the beginning of the pipeline, the system prompts the necessary questions required to fulfill a lab order from a student. Since Backstage already contains information about the student, this data can be leveraged during provisioning to streamline the process. This reduces the amount of information the user must enter and enhances security by preventing incorrect or fabricated inputs. At this point, it is

sufficient to present a menu of available labs. A brief description should also be shown based on the selected lab, ensuring that the user understands what they are ordering.

Clone Repository: The current revision including the full history of the GitOps repository will be cloned by the self-service template using a basic git clone operation. The Isomorphic git package [113] was used to perform git operations in the Node.js environment. The cloned repository serves as the source of truth for the currently active laboratories in the lab engine.

Active Lab Check: File system tools are used to search for a specifically named configuration file within the active-labs folder of the cloned GitOps repository. If a configuration file exists that includes the currently ordering user's name, this action outputs true and provides the file path. Otherwise, it returns false.

Destroy Lab Order: This action checks whether the user has requested to destroy their current lab in the input stage.

Revert Lab Commit: Although this action was intended to use the git revert command, Isomorphic git [113] does not support this functionality. As an alternative, the file path output by the Active Lab Check action is used (if available), and the configuration file is removed from the git index using a git remove operation. Once the file is deleted, the changes are committed, effectively removing the configuration from the repository.

Create New Lab Entry: A new lab configuration is created using a template file containing placeholders for the required values. A text manipulation tool is used to copy the template and fill in the necessary information. This action will also clone a new laboratory guide ticket and assign it to the current user through Jira REST API calls.

Merge Commit: Active changes are merged into the main branch of the GitOps repository using a merge commit operation.

Output: This stage occurs in two scenarios: midway through the template if the student already has an active lab entry in the GitOps repository, or at the end when a new lab entry is created. The output should display the current status of the lab order and a link to a Jira issue containing the next steps in the laboratory exercise.
Finishing a lab exercise: Students will have the ability to decommission their own labs if they finish the exercise before the allotted time, at which point automation will remove the laboratory environment. This measure is implemented to save costs by ensuring that no outdated laboratory environments are hosted in the cluster, preventing potential resource waste and accumulating costs. The lab destruction process can be effectively integrated into the same template as the ordering process by adding a Boolean checkbox that allows students to destroy their current active lab environment. For further development, it would also be beneficial to enable the destruction of a previously active lab while simultaneously ordering a new one. For the sake of simplicity, only two distinct flows are implemented: one for lab creation and one for

lab destruction. The complete flow of the laboratory ordering template is illustrated in Figure 8.

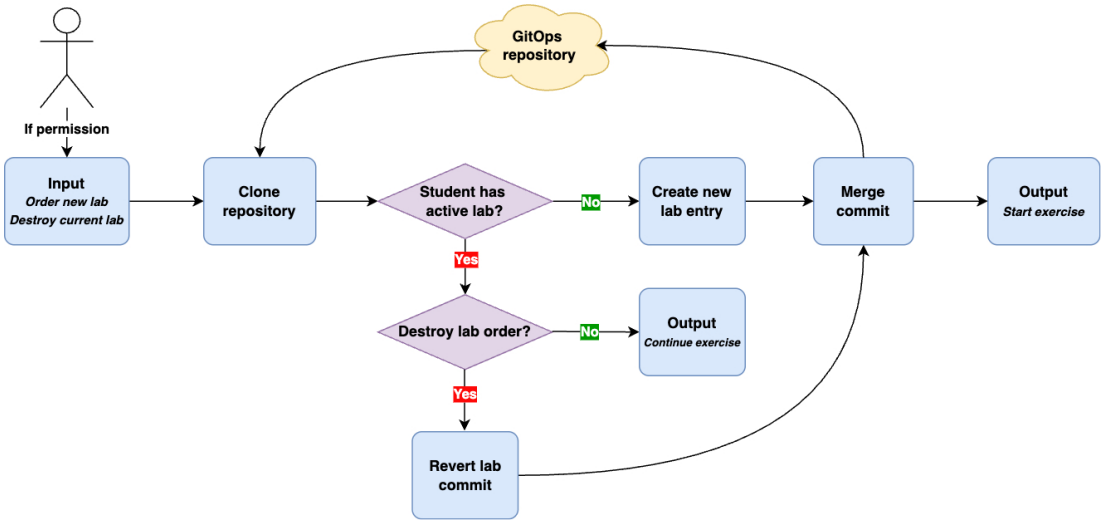


Figure 8. Laboratory order flow in Backstage self-service template.

4.5. Chaos Engineering

The design of the chaos engineering experiment is based on the findings from the failure mode analysis of the reference application, as presented in Appendix 2. The objective is not for students to exploit the most severe attack vectors, but rather to illustrate how common faults occur and demonstrate the resilience of services to recover from them. Students are able to schedule chaos experiments at all times, provided that they already have an active laboratory environment within the engine. Kubernetes RBAC is implemented to ensure that chaos experiments are contained in respective laboratory environments, preventing any impact on other laboratories.

Based on the FMEA of the reference application, a data retrieval latency fault against the database was selected as the target for the guided chaos engineering experiment. This chosen attack vector supports the chosen chaos journey and involves injecting latency faults that directly impact the response times at which the database returns data. By simulating data retrieval delays, the experiment challenges the service to maintain performance and responsiveness under degraded conditions. Notably, the chosen chaos experiment introduces a deliberate, low-severity fault that is ideal for beginners.

4.5.1. Formulating the Hypothesis

Formulating a hypothesis is the first step in the chaos engineering process. This involves defining the system's steady state as the baseline against which deviations are measured during fault injection. For the reference application, this steady state is established by monitoring key metrics that reflect its health from the user's perspective. In the context of ephemeral laboratory exercises, where there is no prior historical data

from the application available, accurately evaluating the steady state of application can be challenging. However, with our simple reference application, the application can be considered healthy if it is available, responsive, and error-free.

Based on these criteria, the following measurable metrics have been defined to represent the steady state of the reference application in an ephemeral laboratory environment.

- **Latency**: The application is considered healthy if the average response time, measured over a short time frame, is less than 100 ms.
- **Error rate**: The application is considered healthy if the rate of failed HTTP requests (here timeouts) stays below the defined threshold.

Hypothesis: If latency faults are injected into the database, then the user interface will either fail to render the data or take longer to do so, leading to a marked increase in the response times of the application. This deviation from the defined steady state will confirm that the injected faults have a tangible impact on system performance.

In the laboratory exercise, this hypothesis will be provided as part of the exercise. By supplying the hypothesis upfront, students can concentrate on experimenting with the chaos tools, rather than formulating their own hypotheses, thereby streamlining the focus on fault injection and observation of the resultant system behavior.

4.5.2. Chaos Experiment

In the guided experiment, students will create a chaos scenario within their own Kubernetes namespace using Chaos Mesh. The objective is to simulate a network fault of type delay, introducing a noticeable latency in the reference application's user interface.

Students will define a fault that injects a 4-second delay into network communications. This delay is deliberately set high, so the impact is clearly observable when interacting with the application. The experiment is scheduled to run for 60 seconds, after which the Chaos Mesh daemon will automatically unschedule the fault, returning the reference application back to its steady state. The effect of steady state and chaos is depicted in Figure 9.

By performing a simple experiment, students gain hands-on experience with chaos engineering principles and learn how to manage, schedule, and observe chaos experiments in a safe environment.

4.5.3. Post Experiment Analysis

After completion of the chaos experiments and observing the service behavior for a few seconds, the next step is to validate the hypothesis. A key consideration is whether the hypothesis holds. This validation is an essential part of the chaos engineering process, as it helps students confirm or refute their assumptions about system behavior under fault conditions. By understanding whether the hypothesis was supported, students

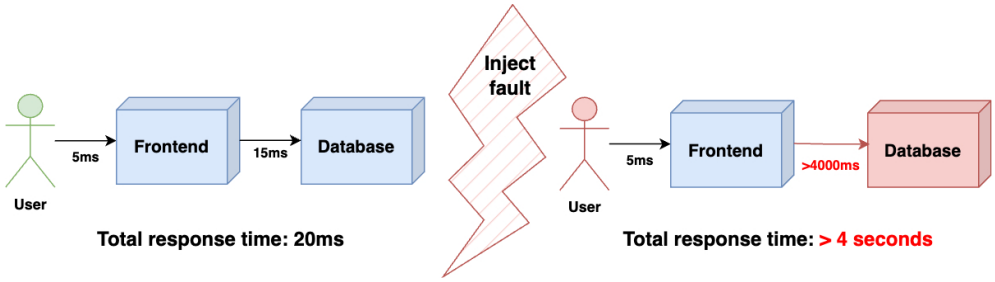


Figure 9. Students will experience fast frontend access but slow database responses, causing delays in graph rendering.

gain valuable insights into the resilience of the system and the effectiveness of the fault injection.

Once the hypothesis has been validated, the formal chaos lab exercise is considered complete. At this point, students are encouraged to further explore and interact with the chaos dashboard, utilizing the available tools within the scope of their assigned RBAC permissions. Although postmortem analysis and the calculation of resiliency scores are valuable practices for deeper investigation, these activities are beyond the scope of this introductory pilot lab. They may, however, be incorporated into more advanced exercises in the future.

4.5.4. Reproducibility of Results

To ensure that each student can consistently reproduce the same chaos engineering faults during the laboratory exercise, a detailed step-by-step guide will be provided. This guide is designed to standardize the learning experience, allowing students to independently replicate the exercise and achieve uniform results in different sessions. The instructions will begin immediately after the ordering process is completed, detailing the entire laboratory exercise, including how to connect to the lab engine, access and navigate the chaos dashboard, inject faults, and experience the resulting behavior firsthand.

The exercise guides were chosen to be provided as Jira tickets, as they contain subtasks that students can check off as they progress through the exercises. Moreover, these tickets are easy to clone, can be assigned to users, and can be retrieved in the future to track the adaptability of the learning platform. The self-service template for lab ordering will be responsible for automating the lab guide cloning process. The steps outlined in the lab guide are visualized in Figure 10.

The example network fault is also delivered as a Kubernetes CR file for the users, in case the user is unable to initiate the chaos experiment from the dashboard. In this way, chaos experiments can be scheduled from the command line, which might even be the preferred option for some users who typically work from the CLI. The 4-second latency fault is defined as a Kubernetes NetworkChaos resource in Appendix 3 Figure 19.

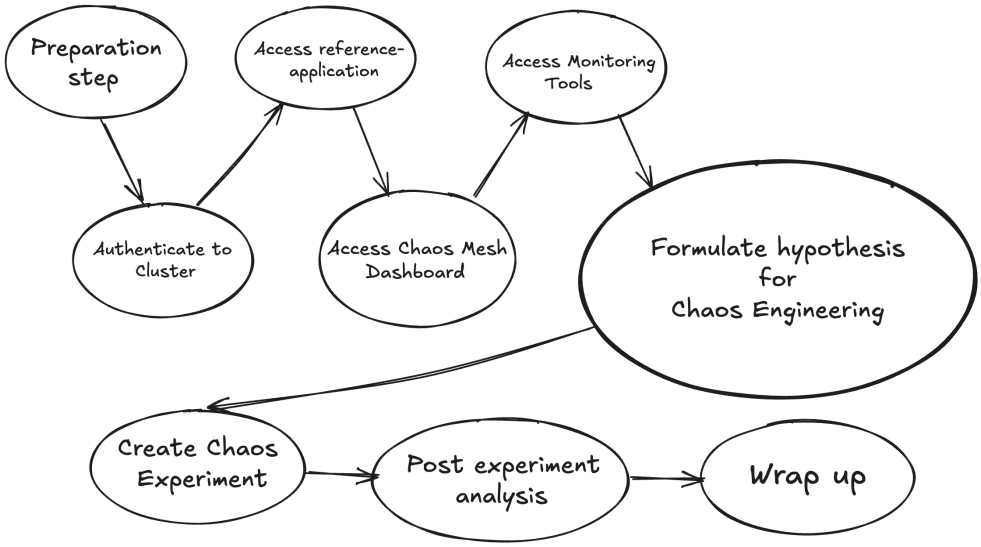


Figure 10. Visual demonstration of steps in the Chaos Lab exercise guide.

4.6. Evaluation

During this section the implemented learning platform will be evaluated through experimental setup in two different perspectives. In the first perspective the learning platform will be evaluated through the piloting laboratory chaos exercise based on its reliability, usability and security in a playtest setting. The platform will be also evaluated based on its performance and scalability with multiple concurrent laboratory exercises in the lab engine. Also the costs of the platform will be approximated and compared to the actual running costs from the playtest. This section provides a comprehensive overview of the evaluation process, including results from the playtest and platform scalability test. The results presented here will later be used to inform the discussion and critically examine the research questions outlined at the beginning of this thesis.

4.6.1. Platform Playtest

A playtest of the implemented learning platform was conducted with members of the Reliability Engineering SIG, as introduced at the beginning of this study. The primary goal of the playtest was to evaluate the reliability, usability, and security of the lab engine by having multiple users simultaneously schedule chaos experiments within a shared learning platform. The playtest was carried out in the Azure cloud, using smaller D2asV5 VM user pool nodes instead of those referenced in Table 6, primarily to reduce costs due to the small user group. The laboratories were provisioned using the self-service template, and the playtest lasted for a duration of four weeks.

During the playtest, a communication channel between the users and platform administrators was established to enable quick action in case of any issues. The users were given individual lab guides to follow during the lab exercise, but after following the guide they were given the freedom to experiment and even try to break

the application. After the users were done with the playtest they were asked to answer a few questions about reliability, usability, and security in their experience during the playtest.

The allocated costs were also monitored throughout the four week playtest and compared against the estimated costs of the designed laboratory platform.

4.6.2. Scalability and Performance

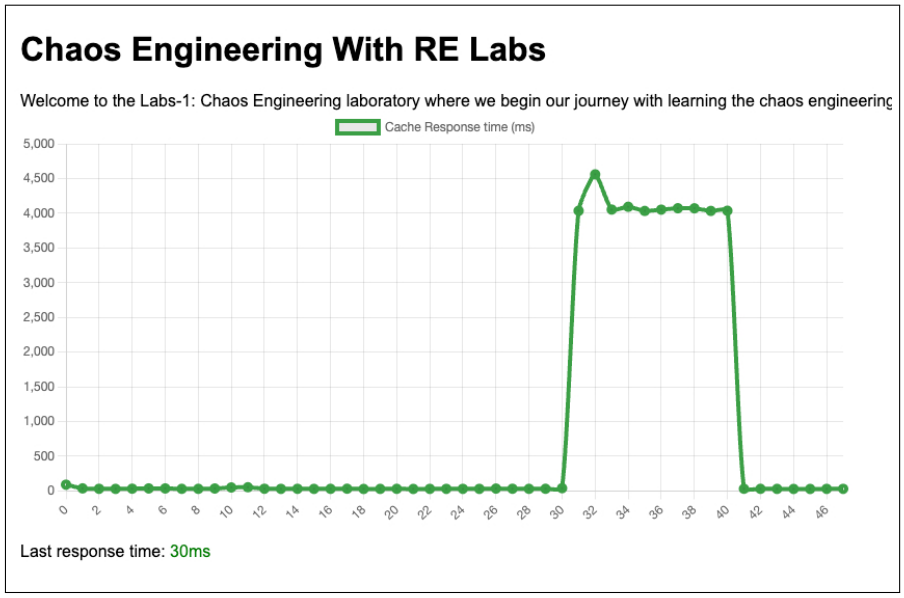
A separate test case was also built to assess the scalability and performance of the laboratory platform using the configuration from Table 6. The performance test involved deploying as many of the chaos exercise laboratory environments as possible, in order to evaluate whether the auto-scaling of the laboratory platform could handle the demand and successfully deploy all environments. A script was created to periodically deploy three laboratory environments at a time, with 60-second intervals between deployments. The environments were deployed using the Helm charts implemented, which are also utilized by ArgoCD for automated deployments. The script was allowed to run until new laboratory environments could no longer be scheduled and were terminally set to a pending state.

The monitoring tools Prometheus [36] and Jaeger [73] were deployed to the lab engine user pool before starting the performance test. The Prometheus scraping configuration used is available in Appendix 4 Figure 20, to gather information on both the laboratory environments and the learning platform itself. All reference applications within laboratory environments were instrumented with OpenTelemetry metrics and tracing. A trace exporter was configured to send trace data to the Jaeger service using the internal Kubernetes URL created for it.

After the monitoring setup was deployed in the lab engine and began receiving telemetry from the laboratory environments, the tests were executed to evaluate the capacity of the implemented learning platform. The results of these tests are discussed in the following chapter.

4.6.3. Results

The platform playtest yielded good results regarding the use of the learning platform for chaos engineering. Students were able to access their laboratory sessions, authentication worked correctly, and authorization allowed students to have the appropriate level of control over both the learning platform and their individual laboratory engines. Chaos experiments were successfully executed against the reference applications within their own laboratory environments, and students did not have access to any other laboratory environments in the engine. The results of running the designed latency fault against the Redis service in the reference application were visible both within the application itself and in the monitoring tools, as depicted in Figure 11. The hypothesis held: the reference application became unusable for four seconds during each page load while the fault was active. After the 60-second fault injection, the response times returned to normal, rendering the application healthy again.



(a)

(b)

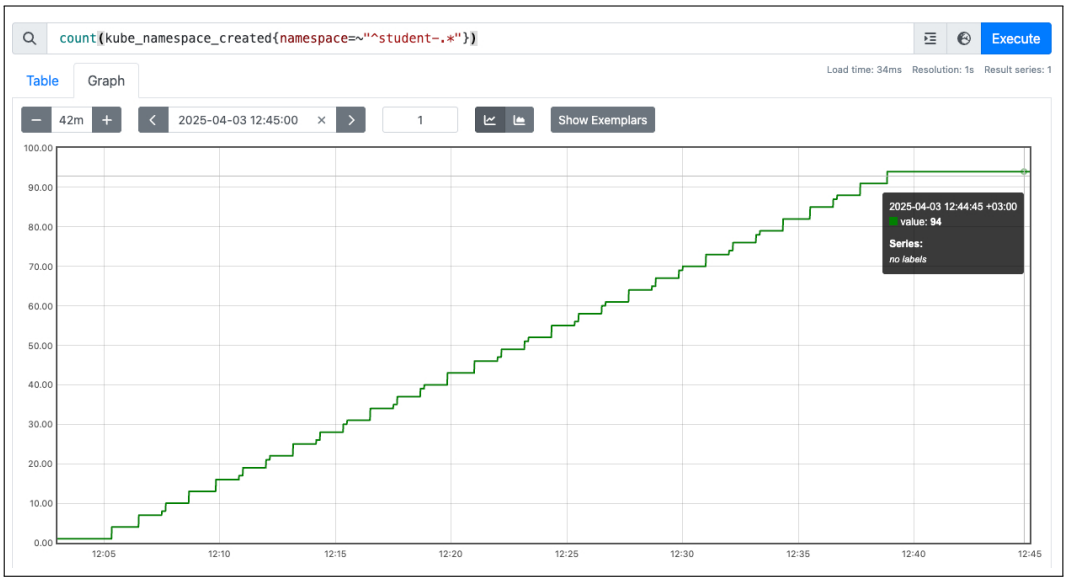
Figure 11. (a) Reference App User Interface during fault injection. (b) Fault injection visibility HTTPS response times in Prometheus. The faults can be immediately seen with precision in monitoring tools.

By examining the traces in Jaeger, it is possible to observe the response time for each service in the call stack, providing detailed insights into how the underlying services operate. In Figure 12, the same trace is presented before and after the fault injection, showing the overall response time of the service as well as the relative time spent in each component of the end-to-end call. In the healthy trace, it is observed that Redis accounts for only one third of the total call stack response time. However, after the fault injection, the Redis response effectively consumes the entire duration of the end-to-end response.

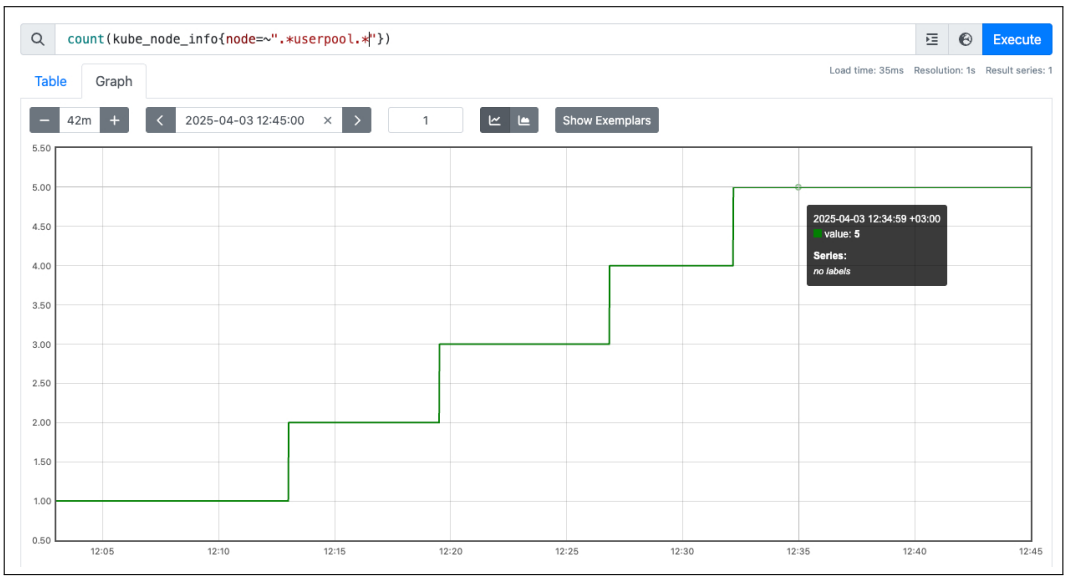
Figure 12. (a) Trace before fault injection, (b) Trace after fault injection. The immediate visibility of changes in monitoring tools demonstrates high observability of the service during fault injection.

Prometheus metrics were used to evaluate the scalability of the learning platform under load, focusing on the number of active laboratory environments and the count of healthy nodes that are running the cluster. These metrics are shown in Figure 13. As the number of running laboratory environments increased during the performance test, it triggers AKS auto-scaling to provision new nodes in response to the growing demand. The figures illustrate a steady rise in active laboratory environments and running user nodes, confirming that the lab engine auto-scaling functions are working as expected. No resource bottlenecks or failures were observed during the performance test, indicating that the platform is capable of scaling effectively and fast to support multiple concurrent laboratory sessions. The number of simultaneously active laboratory environments plateaued at 94, after which no additional pods could be scheduled. CPU and RAM usage on the nodes did not show significant spikes, but scaled in accordance with the resource quotas configured for the laboratory environments. After decommissioning the laboratory environments the amount of nodes dropped due to lower demand on the resources.

The running costs of the learning platform were analyzed using estimates and actual data collected during the playtest. Figure 14 presents multiple approximations of running costs based on different levels of demand in the lab engine. All approximations



(a)



(b)

Figure 13. (a) Prometheus query for namespace count with prefix "student-", indicating the amount of active laboratories. (b) Query showing the number of nodes in the userpool. Together, these illustrate the high scalability of the lab engine.

include a fixed two-node system pool and variable resource costs, which are estimated based on data from the playtest. The figure also explores potential cost savings by introducing scheduled downtimes for the lab engine, in the graph for eight hours during night.

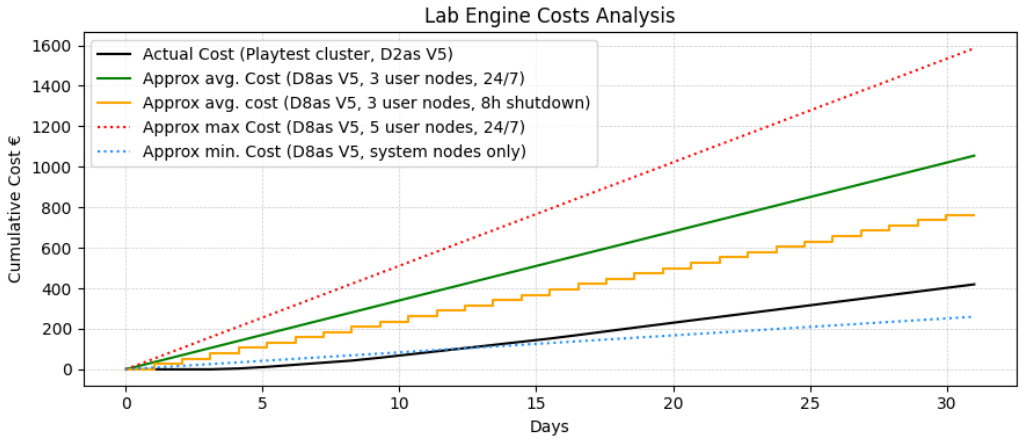


Figure 14. Lab engine cost analysis. All estimations are based on monthly calculations in Microsoft Azure. Additionally an approximation about the costs when shutting down the AKS cluster during nights are also listed.

The actual cost distribution per service during the playtest is shown in Figure 15, providing a clear view of where the majority of the costs originated. Together, these figures offer a practical perspective on the operational expenses associated with maintaining a scalable, cloud native learning platform in the cloud.

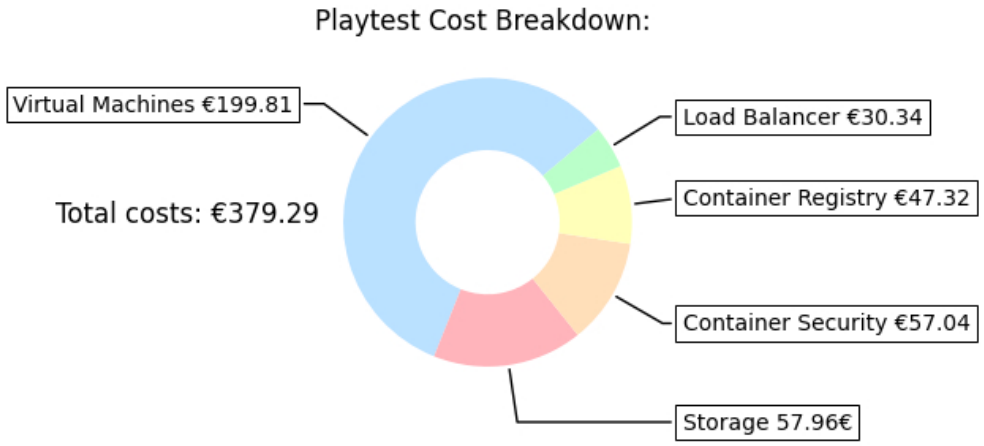


Figure 15. Cost breakdown from individual services in Microsoft Azure during playtest.

5. DISCUSSION

During the discussion, the findings of this study are examined in light of the research questions and relevant previous studies are reviewed. The study involves implementing a cloud native learning platform that utilizes key methodologies and cloud development tools currently popular in the industry. The analysis of usability, reliability, performance, and costs associated with running this platform in the cloud provides a strong reference for the return on investment in designing and building cloud native learning platforms. By comparing these findings with the existing literature on cloud native tools, the study integrates the theoretical and practical implications and outlines directions for future research in this field.

5.1. Revisiting Research Questions and Gap

To begin the discussion, it is important to revisit the research questions and assess what this study set out to accomplish. The aim of this thesis was to examine architectural patterns, tools, and best practices in cloud development to build a scalable, reliable, and secure learning platform that could be delivered as a self-service resource for users. Additionally, the study aimed to analyze how costs accumulate in cloud development and explore strategies to manage them. To support this, an extensive review of relevant literature and industry adoption was conducted in the domains of cloud development and the cloud native tooling landscape. This comprehensive review guided the implementation by providing concrete examples of tools, methodologies, and the latest technologies used in the industry. In this study, the use of cloud-native tools and methodologies to build a scalable platform in the cloud is elaborated upon, contributing to the development of a novel learning platform for designing various types of laboratory exercises based on cloud-native tools.

Cloud native is a relatively new paradigm, and limited academic research is available in this area. This study represents an exploratory step toward understanding the vast landscape of open source and cloud native tooling by analyzing current industry trends and building a novel learning platform. While certain aspects of cloud native—such as containers and orchestration tools like Kubernetes—are well-studied, others like IDPs, GitOps, and observability tools are less represented in academic literature. The findings from this study are interpreted within a broader technical context drawn from the industry. These findings are discussed in this chapter, and the foundations and directions for future research are also proposed.

Within the scope of this thesis, a scalable learning platform was implemented on Microsoft Azure. Before implementation, current trends in cloud development were researched and cloud-native tools based on industry use and CNCF offerings were examined. This extensive research helped guide the implementation by identifying relevant tools, methodologies, and best practices to achieve the best results.

I explored the benefits of open source and cloud native tools in cloud development, with a more detailed analysis of different monitoring tools to determine the best fit for platform observability. The disparity between academic and industry research in the cloud native domain is significant, requiring the use of white papers and documentation from the industry. The CNCF landscape includes hundreds of open source tools that

aligned well with the scope of this thesis, and many were adopted in building the platform. Key tools used include Backstage to enable laboratories as a self-service feature, ArgoCD for automated deployments, OpenTelemetry, Prometheus, and Jaeger for observability, and Chaos Mesh as the introductory tool for the first laboratory exercise.

I investigated the current offerings of hands-on learning platforms, both commercial and open source. Commercial platforms typically support multiple courses tailored to different roles, blending instructional content with practical exercises. However, they often come at a high cost and provide little control over content or infrastructure. Some open source platforms exist, but they mostly focus on Kubernetes and few provide computing resources or sustainability due to lack of sponsorship. No existing commercial or open source platform offered a hands-on learning path for chaos engineering, an emerging methodology for ensuring the resilience of distributed systems. By incorporating elements from these platforms, a hands-on lab exercise that can be ordered through an IDP as a self-service resource was developed. This exercise introduces chaos engineering tooling and is accompanied by a step-by-step lab guide.

Additionally, a self-service software template was created in Backstage to manage the provisioning and decommissioning of lab environments. This self-service approach improves reliability and reduces manual tasks through automation. Backstage's user authentication and permissions framework also ensures controlled access to the ordering templates. The catalog stores user and group information to facilitate straightforward lab ordering.

For the first lab exercise, the emerging discipline of chaos engineering, widely used in cloud systems to test resilience, was studied. Chaos engineering tools from both CNCF and Azure were evaluated, and the first lab was designed using the chaos experiment paradigm (hypothesis, fault injection, and analysis). A step-by-step guide was also created to improve reproducibility. The exercise evaluated the platform's security and reliability by ensuring that users could only impact their own environments during the playtest. This resulted in a novel platform, as no other system currently offers chaos engineering exercises with both tooling and a target application.

5.2. Outcome and Findings

Evaluation of the built platform was done from multiple perspectives. As discussed in Research Question 1, the platform's reliability, security, and usability were evaluated through a playtest with multiple concurrent test users running concurrent fault injection using Chaos Mesh in the learning platform. During the playtest, users were required to follow the step-by-step guide and, after completion, were allowed to explore freely. The learning platform successfully deployed all laboratory environments for playtest users, demonstrating the reliability of the platform. No problems were observed during the playtest from the fault injection or other faulty use cases, indicating that the RBAC configuration shown in Appendix 3 was sufficient to ensure good security control. Usability appeared satisfactory, as users were able to run chaos experiments. However, the scheduling of chaos experiments through the Chaos Dashboard was

deemed inefficient, and other problems with the UI were reported. Nevertheless, scheduling chaos through the CLI presented no issues.

The scalability of the learning platform was evaluated with a performance test that applied continuous load, as discussed in Research Question 1. The performance test allowed experimentation with the practical limits of the node pool architecture of AKS. The results showed good horizontal scalability of the lab engine, effectively spinning up new nodes as resources were exhausted due to the increasing number of concurrent laboratory environments. Effective horizontal down scaling was observed when active laboratories were removed from the lab engine. In the end, 94 concurrent laboratory environments were able to run on the lab engine configuration presented in Table 6, with the resource quotas from Table 5 applied to each laboratory environments. It should be noted that smaller VMs for the node pool and more demanding lab environments result in a smaller number of possible active labs. This test validated the horizontal auto-scaling capabilities of the AKS-based lab engine; however, vertical auto-scaling of the nodes is not possible. Vertical scaling becomes useful if the VMs powering the node pool are deemed too powerful and costly. This highlights the need for observability practices to be in place, enabling developers to continuously monitor the state and adaptability of the lab engine to manage vertical scaling when necessary.

The use of Backstage self-service templates enabled the successful implementation of both the ordering and decommissioning processes for laboratory environments through automation. The template was made available via the running Backstage instance, allowing users to initiate tasks independently. Additionally, adopting GitOps practices with ArgoCD automated the deployment of all laboratory infrastructure. Overall, this approach supported the goal of provisioning the ordering process as an automated, self-service solution, as outlined in Research Question 2. While the outcome was superb, development of the template proved more cumbersome than expected. All actions had to be implemented as custom components, as existing community plugins lacked the features requirements set for the designed template.

To effectively manage costs of running laboratory environments in the cloud, as stated in Research Question 3, some estimates were calculated, and the actual costs from the playtest were compared with these estimations. Cloud providers offer calculators to estimate the running costs of different configurations, and these were used to predict a cost range based on the demand for the lab engine. This enabled a theoretical approach to evaluating the potential costs that could be accumulated with the lab engine. The actual running costs from the playtest were also measured and compared to the estimates. From the breakdown of the playtest, it was observed that resources other than VMs are not as straightforward to estimate. Around two-thirds of the costs were attributed to VMs, and this portion is expected to grow when scaling up the VMs. It was also estimated that costs could be managed by shutting down the AKS cluster during off-hours, such as at night. Cost reductions are shown to scale proportionally with the ratio of the cluster's runtime to downtime—for example, when the cluster is shut down half of the time, VM running costs are cut in half. However, other resources would continue to accumulate costs at roughly the same scale as shown in the breakdown 15.

No cloud provider other than Microsoft Azure was evaluated for hosting the learning platform. However, the cloud native design of the platform does not vendor-lock the platform to only single cloud provider, as it can also be hosted on other clouds or

even on bare metal if desired. Some features of AKS, such as cluster shutdown, are specific to the Azure ecosystem and would need to be implemented other ways if similar functionality is not available by default. It should also be noted that the current implementation relies heavily on Kubernetes as the underlying platform for the laboratory engine, meaning that this study does not extend to other platforms outside the Kubernetes ecosystem.

During the playtest, multiple fixes to the RBAC had to be applied to enable the use of certain Kubernetes tools, such as k9s[9]. These RBAC modifications are reflected in the configuration shown in Appendix 3, Figure 17. Tools other than k9s were not tested against the configured RBAC and may require additional adjustments to function properly.

Some drawbacks were encountered during the implementation phase and during playtesting. Some design elements had to be abandoned because the Git implementation for JavaScript [113] did not support commit reversion. Additionally, the GitHub REST API does not support reverting commits. However, further exploration revealed that the GitHub GraphQL API[10] can revert changes made in a pull request. This functionality could potentially be used for decommissioning laboratory environments, but it was not available for use at the time of implementing the self-service template. The Chaos Mesh Dashboard was found to be highly unstable, and failing to display errors correctly in the UI, resulting in a poor user experience for the playtesters. At times, users were unable to progress through the exercises and received no indication of what was causing the issue.

Furthermore, although proper Kubernetes RBAC can be used to restrict which namespaces users can inject faults into, namespace filtering within the Dashboard interface could not be achieved. While this does not pose a problem with a small number of concurrent laboratory environments, it becomes increasingly difficult to select the correct namespace for chaos experiments as the number of environments grows, since the namespace must be manually chosen from a list in the UI.

Alternatively, running the chaos experiments as Custom Resources and applying them from the command line resulted in no issues. The command-line interface would also interactively prompt the user if any errors were encountered. This supports the assumption that Chaos Mesh is more stable when executed via the command line. One of the requirements for the chaos tools was that they could be run from the command line to support certain developer workflows, and Chaos Mesh fulfills this requirement. The initial laboratory exercise guide includes an example chaos experiment as a Custom Resource in Appendix 3 Figure 19, which can be executed from the command line during the exercise to improve the reproducibility of chaos experiments in the future.

5.3. Future Work

The findings and contributions of this thesis serve as a starting point for future research on how to utilize cloud native tooling when designing platforms that run in the cloud. The cloud native ecosystem shows promising growth and increasing adoption by

[9] https://k9scli.io/

[10] https://docs.github.com/en/graphql/reference/mutations#revertpullrequest

the industry for cloud-based development. Backstage demonstrates great potential in enabling effective self-service processes for developers, reducing the amount of manual intervention. Methodologies like GitOps simplify workflows by allowing familiar development tools such as Git to be used in operations.

As the cloud native landscape has been scarcely researched by academia, a comprehensive survey of the available tools and their adoption across different domains should be conducted. Such a survey could help increase awareness and adoption of cloud native technologies and potentially lead to standardization beyond just the leading edge of the industry.

Although this thesis employs popular and mature CNCF projects to deliver laboratory environments on demand, the open source community offers many more tools that could further optimize the implemented learning platform. Karpenter[11], for example, could be further utilized to scale VM node sizes on the go, and Perses[12] is developing a unified dashboard for displaying telemetry data. The CNCF landscape currently lacks a dedicated Lab-as-a-Service tool, opening opportunities to develop a new project using the tools researched and applied in this thesis.

Currently, the setup is developed on top of a single cloud service provider and for a single cluster. Future research could explore optimizations for deploying laboratory environments across multiple clusters and in multi-cloud or hybrid environments, enabling broader and more resilient delivery. Utilizing cross-cluster networking tools and service meshes like Istio[13] in a multi-cloud setting would enable more reliable services and higher availability. It would also allow the learning platform to remain operational even in the unlikely event of an entire cloud provider outage.

An additional area worth exploring, would be studying how developer experience enhancing IDPs, such as Backstage can be further utilized for different use cases. These platforms are enablers for self-service and efficiency, and other use cases besides the automated deployment of laboratory environments should be further studied.

In addition, enhanced observability capabilities should be explored, including the integration of AI agents into the observability stack to continuously assess the overall health of the platform and laboratories. Combining reactive actions from monitoring tools with AI-driven alerting can increase system reliability and serve as a valuable tool for identifying faults, especially during chaos engineering exercises. For example, the use of Elasticsearch[14] for indexing metric data, applying statistical analysis, or feeding the indexed data to large language models such as Ollama[15] could enable a more thorough examination of fault injections and their effects.

[11] https://karpenter.sh/
[12] https://perses.dev/
[13] https://istio.io/
[14] https://www.elastic.co/elasticsearch
[15] https://ollama.com/

6. SUMMARY

The objective of this thesis was to explore the current architectures and tools used in cloud development, that would enable the development of scalable, reliable, and secure learning platform, that could be operated as self-service, in the cloud. Additionally, the lack of available chaos engineering training environments pushed the study of chaos engineering paradigm, different tools in the field and how a chaos laboratory exercise could be built on the implemented learning platform as the first hands-on exercise laboratory. The primary goal was to do an extensive literature review in best practices in cloud development and how they can be implemented in the architectural decisions in the implemented learning platform. In addition, comprehensive review for cloud native under the CNCF was conducted to explore the most used open source tools in the industry to find the best fitting tools to support scalable learning platform, and enable the ordering of laboratory exercises as a self-service for the users.

As a result, microservice architectures were found to be the best way to deliver cloud services, furthermore containerized laboratories were chosen as they are easy to ship and multiple different orchestration tools exist to manage the lifecycle of ephemeral laboratory environments. Multiple CNCF projects were chosen for the laboratory engine, mainly: Kubernetes, for container orchestration, ArgoCD for automated deployments, OpenTelemetry for holistic observability, Prometheus and Jaeger as monitoring tools, Chaos Mesh as chaos engineering tool and Backstage as the self-service ordering platform.

After implementing the learning platform with said tools, multiple different evaluations were conducted to test the learning platform. Firstly a platform playtest was organized, where multiple concurrent users would progress chaos engineering laboratory exercises in the laboratory engine to evaluate the reliability, usability and security of the built platform. The playtest yielded good results and set access controls were enough to enable progressive learning in users own laboratory environment, but prevented access to any other resource, enabling safe environment for testing and learning, however, the user experience of Chaos Mesh was considered difficult, and caused issues with reproducibility of the hands-on learning labs. In addition, a performance test was conducted, demonstrating that the lab engine can maintain performance under increasing user demand.

The outcomes of this work provide a practical foundation for implementing self-service learning platforms using cloud native technologies, offering valuable insights for both academia and industry into how to leverage cloud native tools in a cloud environment to build such platforms for practical use. This thesis serves as a meaningful contribution to enabling hands-on learning at OP, aiming to build competence with cloud native technologies while continuously enhancing the developer experience.

7. REFERENCES

[1] Gorelik E. (2013) Cloud computing models. Ph.D. thesis, Massachusetts Institute of Technology.

[2] Han R., Guo L., Ghanem M.M. & Guo Y. (2012) Lightweight Resource Scaling for Cloud Applications. In: 2012 12th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (ccgrid 2012), pp. 644–651.

[3] Hofmann P. & Woods D. (2010) Cloud Computing: The Limits of Public Clouds for Business Applications. IEEE Internet Computing 14, pp. 90–93.

[4] Amazon Web Services, AWS Service Level Agreements. https://aws.amazon.com/legal/service-level-agreements/. Accessed: 20 March 2025.

[5] Microsoft, Service Level Agreements for Microsoft Azure. https://www.microsoft.com/licensing/docs/view/Service-Level-Agreements-SLA-for-Online-Services?lang=1. Accessed: 20 March 2025.

[6] Kratzke N. & Quint P.C. (2017) Understanding cloud-native applications after 10 years of cloud computing - A systematic mapping study. Journal of Systems and Software 126, pp. 1–16. URL: https://www.sciencedirect.com/science/article/pii/S0164121217300018.

[7] VMware Inc., Virtualization and Cloud Infrastructure Solutions. https://www.vmware.com/. Accessed: 20 March 2025.

[8] Oracle Corporation, Oracle VM VirtualBox. https://www.virtualbox.org/. Accessed: 20 March 2025.

[9] Chiueh S.N.T.c. & Brook S. (2005) A survey on virtualization technologies. Rpe Report 142.

[10] Casalicchio E. & Iannucci S. (2020) The state-of-the-art in container technologies: Application, orchestration and security. Concurrency and Computation: Practice and Experience 32, p. e5668.

[11] Odun-Ayo I., Ajayi O. & Okereke C. (2017) Virtualization in Cloud Computing: Developments and Trends. In: 2017 International Conference on Next Generation Computing and Information Systems (ICNGCIS), pp. 24–28.

[12] Blenk A., Basta A., Reisslein M. & Kellerer W. (2016) Survey on Network Virtualization Hypervisors for Software Defined Networking. IEEE Communications Surveys & Tutorials 18, pp. 655–685.

[13] Singh A., Korupolu M. & Mohapatra D. (2008) Server-storage virtualization: integration and load balancing in data centers. In: SC'08: Proceedings of the 2008 ACM/IEEE conference on Supercomputing, IEEE, pp. 1–12.

[14] Indu I., Anand P.R. & Bhaskar V. (2018) Identity and access management in cloud environment: Mechanisms and challenges. Engineering Science and Technology, an International Journal 21, pp. 574–588. URL: https://www.sciencedirect.com/science/article/pii/S2215098617316750.

[15] Lorido-Botran T., Miguel-Alonso J. & Lozano J.A. (2014) A review of auto-scaling techniques for elastic applications in cloud environments. Journal of grid computing 12, pp. 559–592.

[16] Roy N., Dubey A. & Gokhale A. (2011) Efficient Autoscaling in the Cloud Using Predictive Models for Workload Forecasting. In: 2011 IEEE 4th International Conference on Cloud Computing, pp. 500–507.

[17] Lins S., Pandl K.D., Teigeler H., Thiebes S., Bayer C. & Sunyaev A. (2021) Artificial intelligence as a service: classification and research directions. Business & Information Systems Engineering 63, pp. 441–456.

[18] Gibson J., Rondeau R., Eveleigh D. & Tan Q. (2012) Benefits and challenges of three cloud computing service models. In: 2012 Fourth International Conference on Computational Aspects of Social Networks (CASoN), pp. 198–205.

[19] Shahrad M., Balkind J. & Wentzlaff D. (2019) Architectural implications of function-as-a-service computing. In: Proceedings of the 52nd annual IEEE/ACM international symposium on microarchitecture, pp. 1063–1075.

[20] Al Shehri W. (2013) Cloud database database as a service. International Journal of Database Management Systems 5, p. 1.

[21] Duan Y., Fu G., Zhou N., Sun X., Narendra N.C. & Hu B. (2015) Everything as a Service (XaaS) on the Cloud: Origins, Current and Future Trends. In: 2015 IEEE 8th International Conference on Cloud Computing, pp. 621–628.

[22] Heroku, Cloud Application Platform. https://www.heroku.com/. Accessed: 20 March 2025.

[23] Microsoft, Microsoft 365 Enterprise Plans and Pricing. https://www.microsoft.com/en-us/microsoft-365/enterprise/microsoft365-plans-and-pricing. Accessed: 20 March 2025.

[24] Amazon Web Services, Aws lambda documentation. https://docs.aws.amazon.com/lambda/. Accessed: 20 March 2025.

[25] Microsoft, Azure functions documentation. https://learn.microsoft.com/en-us/azure/azure-functions/. Accessed: 20 March 2025.

[26] Amazon Web Services, What is a private cloud? https://aws.amazon.com/what-is/private-cloud/.

[27] Singh A. & Chatterjee K. (2017) Cloud security issues and challenges: A survey. Journal of Network and Computer Applications 79, pp. 88–115. URL: https://www.sciencedirect.com/science/article/pii/S1084804516302983.

[28] Briscoe G. & Marinos A. (2009) Digital ecosystems in the clouds: Towards community cloud computing. In: 2009 3rd IEEE International Conference on Digital Ecosystems and Technologies, pp. 103–108.

[29] Gartner (2024), Gartner Forecasts Worldwide Public Cloud End-User Spending to Total $723 Billion in 2025. https://www.gartner.com/en/newsroom/press-releases/2024-11-19-gartner-forecasts-worldwide-public-cloud-end-user-spending-to-total-723-billion-dollars-in-2025. Accessed: 24 March 2025.

[30] Microsoft (2022), Microsoft announces intent to build a new datacenter region in Finland, accelerating sustainable digital transformation and enabling large-scale carbon-free district heating. https://news.microsoft.com/europe/2022/03/17/microsoft-announces-intent-to-build-a-new-datacenter-region-in-finland-accelerating-sustainable-digital-transformation-and-enabling-large-scale-carbon-free-district-heating/. Accessed: 24 March 2025.

[31] Kondense, Kubernetes Cost Visibility and Optimization. https://github.com/unagex/kondense. Accessed: 23-03-2025.

[32] Baarzi A.F. & Kesidis G. (2021) Showar: Right-sizing and efficient scheduling of microservices. In: Proceedings of the ACM Symposium on Cloud Computing, ACM, pp. 427–441.

[33] Amazon Web Services, AWS Cost Explorer. https://aws.amazon.com/aws-cost-management/aws-cost-explorer/. Accessed: 24 March 2025.

[34] Microsoft Azure, Azure advisor overview. https://learn.microsoft.com/en-us/azure/advisor/advisor-overview. Accessed: 27 April 2025.

[35] Tatineni S. (2019) Cost Optimization Strategies for Navigating the Economics of AWS Cloud Services. International Journal of Advanced Research in Engineering and Technology (IJARET) 10, pp. 827–842.

[36] Chen L., Xian M. & Liu J. (2020) Monitoring system of openstack cloud platform based on prometheus. In: 2020 International Conference on Computer Vision, Image and Deep Learning (CVIDL), pp. 206–209.

[37] Deng S., Zhao H., Huang B., Zhang C., Chen F., Deng Y., Yin J., Dustdar S. & Zomaya A.Y. (2024) Cloud-Native Computing: A Survey From the Perspective of Services. Proceedings of the IEEE 112, pp. 12–46.

[38] Wiggins A., The Twelve-Factor App. https://12factor.net/. Accessed: 20 March 2025.

[39] Cloud Native Computing Foundation (2024), Cloud Native Definition. https://www.cncf.io/about/. Accessed: 20 March 2025.

[40] Cloud Native Computing Foundation, KCD Helsinki - Kubernetes Community Days. https://community.cncf.io/kcd-helsinki/. Accessed: 20 March 2025.

[41] Blinowski G., Ojdowska A. & Przybyłek A. (2022) Monolithic vs. Microservice Architecture: A Performance and Scalability Evaluation. IEEE Access 10, pp. 20357–20374.

[42] Hasselbring W. & Steinacker G. (2017) Microservice architectures for scalability, agility and reliability in e-commerce. In: 2017 IEEE International Conference on Software Architecture Workshops (ICSAW), pp. 243–246.

[43] Francesco P.D., Malavolta I. & Lago P. (2017) Research on architecting microservices: Trends, focus, and potential for industrial adoption. In: 2017 IEEE International Conference on Software Architecture (ICSA), pp. 21–30.

[44] Espe L., Jindal A., Podolskiy V. & Gerndt M. (2020) Performance Evaluation of Container Runtimes. In: CLOSER, pp. 273–281.

[45] Howard M. & Bruce Irvin R. (2023) Qemu emulation within a docker container. In: K. Arai (ed.) Proceedings of the Future Technologies Conference (FTC) 2023, Volume 3, Springer Nature Switzerland, Cham, pp. 63–80.

[46] Harve B.M., Bidkar D.M., Krishnappa M.S., Pandy G., Jayaram V., Veerapaneni P.K. & Mehta G. (2024) The Cloud-Native Revolution: Microservices in a Cloud-Driven World. In: 2024 International Conference on Intelligent Cybernetics Technology & Applications (ICICyTA), pp. 1043–1048.

[47] Pan Y., Chen I., Brasileiro F., Jayaputera G. & Sinnott R. (2019) A Performance Comparison of Cloud-Based Container Orchestration Tools. In: 2019 IEEE International Conference on Big Knowledge (ICBK), pp. 191–198.

[48] Khan A. (2017) Key Characteristics of a Container Orchestration Platform to Enable a Modern Application. IEEE Cloud Computing 4, pp. 42–48.

[49] Jawarneh I.M.A., Bellavista P., Bosi F., Foschini L., Martuscelli G., Montanari R. & Palopoli A. (2019) Container Orchestration Engines: A Thorough Functional and Performance Comparison. In: ICC 2019 - 2019 IEEE International Conference on Communications (ICC), pp. 1–6.

[50] Malviya A. & Dwivedi R.K. (2022) A Comparative Analysis of Container Orchestration Tools in Cloud Computing. In: 2022 9th International Conference on Computing for Sustainable Global Development (INDIACom), pp. 698–703.

[51] Cloud Native Computing Foundation, CNCF Cloud Native Landscape - Scheduling & Orchestration. https://landscape.cncf.io/guide#orchestration-management--scheduling-orchestration. Accessed: 25 March 2025.

[52] Amazon Web Services, Amazon Elastic Kubernetes Service. https://aws.amazon.com/eks/. Accessed: 25 March 2025.

[53] Microsoft Azure, Azure Kubernetes Service. https://azure.microsoft.com/en-us/products/kubernetes-service. Accessed: 25 March 2025.

[54] Amazon Web Services, Amazon Elastic Container Service. https://aws.amazon.com/ecs/. Accessed: 25 March 2025.

[55] Microsoft Azure, Azure container instances. https://azure.microsoft.com/en-us/products/container-instances. Accessed: 25 March 2025.

[56] Reddy Y.S.D., Reddy P.S., Ganesan N. & Thangaraju B. (2022) Performance Study of Kubernetes Cluster Deployed on Openstack,VMs and BareMetal. In: 2022 IEEE International Conference on Electronics, Computing and Communication Technologies (CONECCT), pp. 1–5.

[57] Muddinagiri R., Ambavane S. & Bayas S. (2019) Self-Hosted Kubernetes: Deploying Docker Containers Locally With Minikube. In: 2019 International Conference on Innovative Trends and Advances in Engineering and Technology (ICITAET), pp. 239–243.

[58] Deepa N., Prabadevi B., Krithika L. & Deepa B. (2020) An analysis on Version Control Systems. In: 2020 International Conference on Emerging Trends in Information Technology and Engineering (ic-ETITE), pp. 1–9.

[59] Zolkifli N.N., Ngah A. & Deraman A. (2018) Version Control System: A Review. Procedia Computer Science 135, pp. 408–415. URL: https://www.sciencedirect.com/science/article/pii/S1877050918314819, the 3rd International Conference on Computer Science and Computational Intelligence (ICCSCI 2018) : Empowering Smart Technology in Digital Era for a Better Life.

[60] Brindescu C., Codoban M., Shmarkatiuk S. & Dig D. (2014) How do centralized and distributed version control systems impact software changes? In: Proceedings of the 36th International Conference on Software Engineering, ICSE 2014, Association for Computing Machinery, New York, NY, USA, p. 322–333. URL: https://doi.org/10.1145/2568225.2568322.

[61] Rodríguez-Bustos C. & Aponte J. (2012) How Distributed Version Control Systems impact open source software projects. In: 2012 9th IEEE Working Conference on Mining Software Repositories (MSR), pp. 36–39.

[62] Mittal P. & Narang P. (2023) Performance assessment and analysis of development and operations based automation tools for source code management. International Journal of Electrical and Computer Engineering 13, p. 1817.

[63] Weaveworks and CNCF (2024), Gitops: Operations by pull request. https://www.gitops.tech/. URL: https://www.gitops.tech/, accessed: 25 March 2025.

[64] Beetz F. & Harrer S. (2022) Gitops: The evolution of devops? IEEE Software 39, pp. 70–75.

[65] Niedermaier S., Koetter F., Freymann A. & Wagner S. (2019) On observability and monitoring of distributed systems – an industry interview study. In: S. Yangui, I. Bouassida Rodriguez, K. Drira & Z. Tari (eds.) Service-Oriented Computing, Springer International Publishing, Cham, pp. 36–52.

[66] Haselböck S. & Weinreich R. (2017) Decision Guidance Models for Microservice Monitoring. In: 2017 IEEE International Conference on Software Architecture Workshops (ICSAW), pp. 54–61.

[67] Usman M., Ferlin S., Brunstrom A. & Taheri J. (2022) A Survey on Observability of Distributed Edge & Container-Based Microservices. IEEE Access 10, pp. 86904–86919.

[68] Li S., Zhang H., Jia Z., Zhong C., Zhang C., Shan Z., Shen J. & Babar M.A. (2021) Understanding and addressing quality attributes of microservices architecture: A Systematic literature review. Information and Software Technology 131, p. 106449. URL: https://www.sciencedirect.com/science/article/pii/S0950584920301993.

[69] Waseem M., Liang P., Shahin M., Di Salle A. & Márquez G. (2021) Design, monitoring, and testing of microservices systems: The practitioners' perspective. Journal of Systems and Software 182, p. 111061. URL: https://www.sciencedirect.com/science/article/pii/S0164121221001588.

[70] Rios J., Jha S. & Shwartz L. (2022) Localizing and Explaining Faults in Microservices Using Distributed Tracing. In: 2022 IEEE 15th International Conference on Cloud Computing (CLOUD), pp. 489–499.

[71] OpenTelemetry (2024), OpenTelemetry Documentation. URL: https://opentelemetry.io/docs/, Accessed: 28 March 2025.

[72] Microsoft Azure (2025), Azure Monitor OpenTelemetry-based Exporter. https://learn.microsoft.com/en-us/azure/azure-monitor/app/opentelemetry. URL: https://learn.microsoft.com/en-us/azure/azure-monitor/app/opentelemetry, Accessed: 26 March 2025.

[73] Jaeger Tracing (2024), Jaeger: open source, end-to-end distributed tracing. URL: https://www.jaegertracing.io/, Accessed: 28 March 2025.

[74] Liu H.C., Liu L. & Liu N. (2013) Risk evaluation approaches in failure mode and effects analysis: A literature review. Expert Systems with Applications 40, pp. 828–838. URL: https://www.sciencedirect.com/science/article/pii/S0957417412009712.

[75] Grafana Labs (2025), Grafana Documentation. https://grafana.com/docs/grafana/latest/. URL: https://grafana.com/docs/grafana/latest/, Accessed: 26 March 2025.

[76] Cortex Metrics (2024), Cortex Documentation. URL: https://cortexmetrics.io/docs/, Accessed: 28 March 2025.

[77] Thanos Project (2024), Thanos - Highly available Prometheus setup with long term storage capabilities. URL: https://thanos.io/, Accessed: 28 March 2025.

[78] Anteon (2024), Anteon Documentation. URL: https://getanteon.com/docs/, Accessed: 28 March 2025.

[79] BotKube (2024), BotKube Documentation. URL: https://docs.botkube.io/, Accessed: 28 March 2025.

[80] Microsoft (2025), Azure Monitor. https://learn.microsoft.com/en-us/azure/azure-monitor/. Accessed: 31 March 2025.

[81] Datadog (2024), Datadog Observability Platfor. URL: https://www.datadoghq.com/monitoring/observability-platform/, Accessed: 28 March 2025.

[82] Dynatrace (2024), Application Observability. URL: https://www.dynatrace.com/platform/application-observability/, Accessed: 28 March 2025.

[83] Fluent Bit (2024), Fast and Lightweight Logs and Metrics Processor. URL: https://fluentbit.io/, Accessed: 28 March 2025.

[84] Xu L., Huang D. & Tsai W.T. (2014) Cloud-Based Virtual Laboratory for Network Security Education. IEEE Transactions on Education 57, pp. 145–150.

[85] Microsoft, Azure Lab Services. URL: https://azure.microsoft.com/en-us/products/lab-services/, Accessed: 24 February 2025.

[86] Roman S., Labs4Grabs Documentation. https://docs.labs4grabs.io. Accessed: 27 March 2025.

[87] Katacoda, Interactive Learning Scenarios. https://github.com/katacoda. Accessed: 27 March 2025.

[88] Collabnix, KubeLabs - Kubernetes Hands-on Labs. https://collabnix.github.io/kubelabs/. Accessed: 27 March 2025.

[89] KodeKloud, DevOps and Cloud Learning Platform. https://kodekloud.com/. Accessed: 27 March 2025.

[90] Spotify, Backstage Documentation – Software Templates. https://backstage.io/docs/features/software-templates/. Accessed: 27 March 2025.

[91] Spotify, Backstage Documentation – Software Catalog. https://backstage.io/docs/features/software-catalog/. Accessed: 27 March 2025.

[92] Spotify, Backstage Documentation – Authentication. https://backstage.io/docs/auth/. Accessed: 27 March 2025.

[93] Spotify, Github Repository for Backstage Community Plugins. https://github.com/backstage/community-plugins. Accessed: 27 March 2025.

[94] Simonsson J., Zhang L., Morin B., Baudry B. & Monperrus M. (2021) Observability and chaos engineering on system calls for containerized applications in docker. Future Generation Computer Systems 122, pp. 117–129. URL: https://www.sciencedirect.com/science/article/pii/S0167739X21001163.

[95] Jernberg H., Runeson P. & Engström E. (2020) Getting started with chaos engineering - design of an implementation framework in practice. In: Proceedings of the 14th ACM / IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM), ESEM '20, Association for Computing Machinery, New York, NY, USA. URL: https://doi.org/10.1145/3382494.3421464.

[96] Microsoft Azure (2024), Microsoft azure. https://azure.microsoft.com/. Accessed: 27 March 2025.

[97] Poltronieri F., Tortonesi M. & Stefanelli C. (2022) A chaos engineering approach for improving the resiliency of it services configurations. In: NOMS 2022-2022 IEEE/IFIP Network Operations and Management Symposium, pp. 1–6.

[98] Cabanes B., Hubac S., Le Masson P. & Weil B. (2021) Improving reliability engineering in product development based on design theory: the case of fmea in the semiconductor industry. Research in Engineering Design 32, pp. 309–329.

[99] Dalal S. & Chhillar R.S. (2013) Empirical study of root cause analysis of software failure. SIGSOFT Softw. Eng. Notes 38, p. 1–7. URL: https://doi.org/10.1145/2492248.2492263.

[100] Lehtinen T.O., Virtanen R., Viljanen J.O., Mäntylä M.V. & Lassenius C. (2014) A tool supporting root cause analysis for synchronous retrospectives in distributed software teams. Information and Software Technology 56, pp. 408–437. URL: https://www.sciencedirect.com/science/article/pii/S0950584914000159.

[101] Murphy N.R., Beyer B., Jones C. & Petoff J. (2016) Site Reliability Engineering: How Google Runs Production Systems. O'Reilly Media.

[102] Metsa J., Katara M. & Mikkonen T. (2007) Testing non-functional requirements with aspects: An industrial case study. In: Seventh International Conference on Quality Software (QSIC 2007), pp. 5–14.

[103] Microsoft (2024), Azure Kubernetes Service. URL: https://learn.microsoft.com/en-us/azure/aks, Accessed: 24 February 2025.

[104] Microsoft (2024), App Service Documentation. URL: https://learn.microsoft.com/en-us/azure/app-service/, Accessed: 24 February 2025.

[105] Kubernetes Contributors (2025), Stateless Application Example: Guestbook. URL: https://kubernetes.io/docs/tutorials/stateless-application/guestbook, Accessed: 25 March 2025.

[106] FluxCD Project (2025), Flux: The GitOps Family of Projects. https://fluxcd.io/. URL: https://fluxcd.io/, Accessed: 26 March 2025.

[107] Codefresh (2025), Weaveworks Shuts Down Legacy and Next Steps for GitOps Users. https://codefresh.io/learn/weaveworks-shuts-down-legacy-and-next-steps/. URL: https://codefresh.io/learn/weaveworks-shuts-down-legacy-and-next-steps/, Accessed: 26 March 2025.

[108] Argo CD Project (2025), Argo CD Documentation. https://argo-cd.readthedocs.io/en/stable/. URL: https://argo-cd.readthedocs.io/en/stable/, Accessed: 26 March 2025.

[109] Microsoft Docs (2025), Azure Chaos Studio. URL: https://learn.microsoft.com/en-us/azure/chaos-studio/chaos-studio-overview, Accessed: 25 March 2025.

[110] Chaos Toolkit (2025), Azure Driver Chaos Toolkit. URL: https://chaostoolkit.org/drivers/azure/, Accessed: 25 March 2025.

[111] Atlassian, Jira Software. https://www.atlassian.com/software/jira. Accessed: 31 March 2025.

[112] Microsoft, Use system and user node pools in Azure Kubernetes Service. https://learn.microsoft.com/en-us/azure/aks/use-system-pools?tabs=azure-cli. Accessed: 27 April 2025.

[113] Isomorphic-git Contributors, Isomorphic-git. URL: https://isomorphic-git.org/, Accessed: 31 March 2025.

8. APPENDICES

Appendix 1 AKS configuration for System and User Nodes
Appendix 2 FMEA table for reference application
Appendix 3 Helm charts for laboratory environment
Appendix 4 Monitoring configuration
Appendix 5 ArgoCD configurations

Table 6. Overview of AKS Node Configurations for System and User Nodes.

Node	System Nodes	User Nodes
OS	Linux	Linux
Runtime	containerd	containerd
VM Size	Standard_D4as_v5	Standard_D8as_v5
vCPUs	4	8
RAM	16GB	32GB
Storage	Shared 50GB	
Min n. nodes	2	0
Max n. nodes	3	5

Service	Failure mode	Effect	Severity
Frontend Service	User interface is not available	Users can not use the whole application	10
Backend Service	Database connection failure	Backend fails to connect to database and can not serve the requests to frontend	9
Frontend Service	Reverse Proxy routing failure	Graph is out of sync and fails to get new response times to render	8
Backend Service	Backend service unavailable	Users get no critical data from backend and are server error messages	8
Backend Service	API Response Timeout	Backend can not serve responses in timely manner, causing timeouts in frontend	8
Database Service	Data loss	Data may be lost or become corrupted impacting on reliability of reference application	8
Database Service	Database unavailable	Backend needs to serve possible old data or no data at all	8
All Services	Replication failure	Multiple replicas are out-of-sync and return unreliable values	7
All Services	High CPU Utilization	Gradual CPU consumption can slow down the service	7
All Services	Memory leak	Gradual memory increase will eventually lead to process crash and memory kill	6[a]
Frontend Service	Asset loading failure	Images, fonts and other files fail to load, broken user interface	6
Frontend Service	Slow user interface	The reference application is unresponsive and lead to bad user-experience	6
Database Service	**Data retrieval latency**	**Database operations take long time and lead to delayed requests to frontend**	**6**[b]

Table 7. This table lists the potential failure modes across all micro-services in the reference application. The modes are ranked from highest to lowest severity as perceived by the user.

[a] Depends heavily on the replica settings in the deployment.

[b] The chosen failure mode for pilot chaos engineering experiment exercise.

Figure 16. Helm template to generate a service account with sufficient access for each user to authenticate with Chaos Mesh.

```
apiVersion: v1
kind: ServiceAccount
metadata:
    namespace: {{ .Values.ns }}
    name: account-relab-manager
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
    namespace: {{ .Values.ns }}
    name: role-relab-manager
rules:
  - apiGroups: ['chaos-mesh.org']
    resources: ['*']
    verbs: ['get', 'list', 'watch', 'create',
    'delete', 'patch', 'update']
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
    name: bind-relab-manager
    namespace: {{ .Values.ns }}
subjects:
  - kind: ServiceAccount
    name: account-relab-manager
    namespace: {{ .Values.ns }}
roleRef:
    kind: Role
    name: role-relab-manager
```

Figure 17. Kubernetes RBAC for students to perform certain actions in their own namespace.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
    namespace: {{ .Values.ns }}
    name: relab-student-role
rules:
  - apiGroups: ['*']
    resources: ['*']
    verbs: ['get', 'list', 'create', 'watch']
  - apiGroups: ['apps']
    resources: ['*']
    verbs: ['get', 'list', 'create']
  - apiGroups: ['', 'networking.k8s.io', 'networking', 'extensions']
    resources: ['ingresses']
    verbs: ['get', 'watch', 'list']
  - apiGroups: ['chaos-mesh.org']
    resources: ['*']
    verbs: ['get', 'list', 'watch', 'create', 'delete', 'patch', '
       update']
  - apiGroups: ['']
    resources: ['serviceaccounts/token']
    verbs: ['create']
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
    name: relab-student
    namespace: {{ .Values.ns }}
subjects:
  - kind: User
    name: {{ .Values.admAccount }}
    apiGroup: rbac.authorization.k8s.io
roleRef:
    kind: Role
    name: relab-student-role
    apiGroup: rbac.authorization.k8s.io
```

Figure 18. Cluster role and role binding configuration for students to list namespaces in cluster.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
    name: ns-lister
rules:
  - apiGroups: ['']
    resources: ['namespaces']
    verbs: ['get', 'list', 'watch']
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
    name: list-namespaces
subjects:
# List all users with active laboratory in the lab engine
{{- range .Values.Users }}
  - kind: User
    name: {{ .userPrincipalName }}
    apiGroup: rbac.authorization.k8s.io
{{- end }}
roleRef:
      kind: ClusterRole
      name: ns-lister
    apiGroup: rbac.authorization.k8s.io
```

Figure 19. Example 4-second latency fault for Redis pods as Kubernetes Custom Resource.

```
# An example Networking fault to redis pod lasting 60 seconds
apiVersion: chaos-mesh.org/v1alpha1
kind: NetworkChaos
metadata:
    name: redis-network-fault
    namespace: {{ .Values.ns }}
spec:
    action: delay
    mode: one
    selector:
        labelSelectors:
            app: redis
    delay:
        latency: '4s'
        correlation: '0'
        jitter: '0'
    duration: '60s'
```

Figure 20. Configuration file for Prometheus to scrape metrics from laboratory environments, Kubernetes cluster and Prometheus itself.

```
global:
  scrape_interval: 10s
  evaluation_interval: 10s
  scrape_configs:
  - job_name: 'prometheus'
    static_configs:
    - targets: ['localhost:9090']
    - job_name: 'kube-state-metrics'
      static_configs:
        - targets: ['kube-state-metrics.default\\.svc.cluster.local
            :8080']
    - job_name: 'laboratory-environment-pods'
      honor_labels: true
      kubernetes_sd_configs:
      - role: pod
        relabel_configs:
        - source_labels: [
            __meta_kubernetes_pod_annotation_prometheus_io_scrape]
            action: keep
            regex: true
        - source_labels: [
            __meta_kubernetes_pod_annotation_prometheus_io_path]
            action: replace
            target_label: __metrics_path__
            regex: (.+)
        - source_labels: [__address__,
            __meta_kubernetes_pod_annotation_prometheus_io_port]
            action: replace
            regex: ([^:]+)(?::\d+)?;(\d+)
            replacement: $1:$2
            target_label: __address__
        - action: labelmap
            regex: __meta_kubernetes_pod_label_(.+)
          - source_labels: [__meta_kubernetes_namespace]
            action: replace
            target_label: kubernetes_namespace
          - source_labels: [__meta_kubernetes_pod_name]
            action: replace
            target_label: kubernetes_pod_name
```

Figure 21. Instrumentation file for using OpenTelemetry with Node.js application, including exporters to Prometheus and Jaeger.

```
/* OpenTelemetry instrumentation */
const labId = process.env['lab.id'] || 'defaultId'
const labNamespace = process.env['lab.namespace'] || 'namespace'
const studentId = process.env['lab.id'] || 'studentId'

const resource = resourceFromAttributes({
    [ATTR_SERVICE_NAME]: 'reference-app-backend',
    'lab.namespace': labNamespace,
    'student.id': studentId,
    'lab.id': labId
  });
const traceExporter = new OTLPTraceExporter(
    {
        // KubeDNS resolves this to the Jaeger service
        url: 'jaeger.jaeger.svc.cluster.local:4317',
        credentials: ChannelCredentials.createInsecure()
    }
  );
const spanProcessor = new BatchSpanProcessor(traceExporter);
const metricReader = new PrometheusExporter({ port: 9464 });

const sdk = new NodeSDK({
    resource,
    traceExporter,
    metricReader,
    spanProcessor,
    instrumentations: [getNodeAutoInstrumentations()],
  });

sdk.start();
```

Figure 22. ArgoCD ApplicationSet configuration for an example laboratory.

```yaml
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata:
  name: laboratories
  namespace: argocd
spec:
  goTemplate: true
  goTemplateOptions: ['missingkey=error']
  generators:
    - git:
        repoURL: // URL to GitOps repository
        revision: main
        files:
          - path: 'active-labs/*-chaos-laboratory.json'
  template:
    metadata:
      name: '{{.lab.id}}-{{.student.namespace}}'
      namespace: argocd
    spec:
      project: chaos-laboratory
      source:
        repoURL: // URL to GitOps repository
        targetRevision: HEAD
        path: 'helm-charts/{{.laboratory.id}}'
        helm:
          namespace: '{{.student.namespace}}'
          parameters:
          - name: 'namespace'
            value: '{{.student.namespace}}'
          - name: 'email'
            value: '{{.student.email}}'
      syncPolicy:
        automated:
          prune: true // Automatically delete laboratory
          ↪  environments
          selfHeal: true
      destination:
        server: https://kubernetes.default.svc
        namespace: '{{.student.namespace}}'
```

Figure 23. Kubernetes role-based access rights for students in shared learning platform.

```
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata:
  name: cluster-rbac
  namespace: argocd
spec:
  goTemplate: true
  goTemplateOptions: ['missingkey=error']
  generators:
    - git:
        repoURL: // URL to GitOps repository
        revision: main
        files:
          - path: 'active-labs/*.json' // All labs
  template:
    metadata:
      name: '{{path.basename}}-rbac'
      namespace: argocd
    spec:
      project: cluster-wide-student-rbac
      source:
        repoURL: // URL to GitOps repository
        targetRevision: HEAD
        path: 'helm-charts/cluster-rbac'
      destination:
        server: 'https://kubernetes.default.svc'
        namespace: default
      syncPolicy:
        automated:
          prune: true
          selfHeal: true
```